

Document Intelligence Workshop - Claims

[Document Intelligence Documentation](#)

[Workshop Github Repo](#)

For this workshop we'll be simulating working with data for a Health Claims Processing use case. We have structured data from our "relational" system, for 100 Claims. Information here includes data on the Patient, Claim, Provider, Payer, Procedure, and Diagnosis.

Along with this structured data, we have unstructured data in the form of doctors' notes. These are pdf files that were stored in our SharePoint system that are related to these claims.

Currently a person has to manually examine the notes in relation to the structured data to validate that procedure and diagnosis codes in the notes match what was filed in the claim. Processing can take time and sometimes as a result, errors are made. By connecting the data in Neo4j and leveraging LLMs, we can accelerate and automate the review process, providing value in time and money to the business.

In this workshop we'll use Document Intelligence to:

1. Create a data model for our pdf files.
2. Extract the doctor's notes text from our pdf files in import them into the data model.

Next, we'll use LOAD CSV to

3. load our structured data and connect it with the unstructured.

In this way you'll see how Neo4j is used to connect the unstructured and structured. How we can append to an existing graph and continue to enrich it.

We then Query the data for GraphRAG

4. Querying the text embeddings extracted from the unstructured data for semantic similarity search
5. Query the text embeddings along with relationships and other connected nodes to perform VectorCypher retrieval for GraphRag (more powerful than RAG)

Finally, we'll use Aura Agent

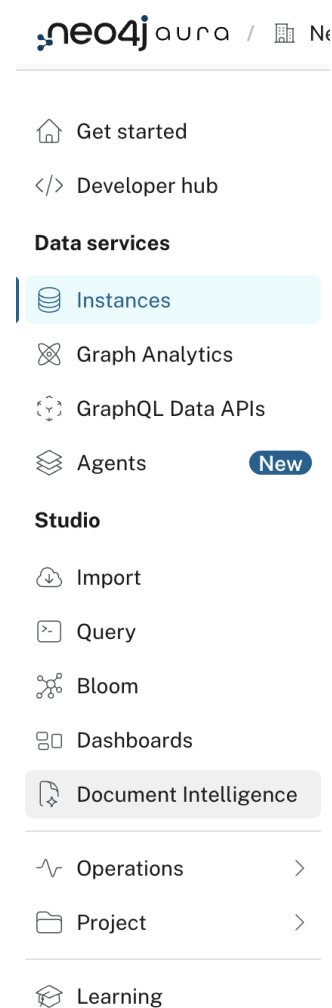
6. To create an agent with tools so we can use natural language to interact with the graph we've created.

Let's begin by going to our Aura Free instances and loading our pdf files using Document Intelligence.

Load Unstructured Data

The data for this section can be found in the /data directory of the [Workshop Github Repository](#). Download this repo as a .zip or using git.

1. Create the Data Model



Navigate to “Document Intelligence” in the left Aura menu pane and select it.

For this exercise, we're just going to create the data model for 1 document and then extract our document into the model. The Document Intelligence feature is currently in preview and works for up to 20 documents of reasonable page length. Processing several documents can take several minutes, so we will go through the process to create the model for one document and upload it. Then, in the interest of time as well as consistency, we will restore our database with modeled and chunked documents that have been processed for us.

In short, the process for importing unstructured data into a knowledge graph consists of five steps:

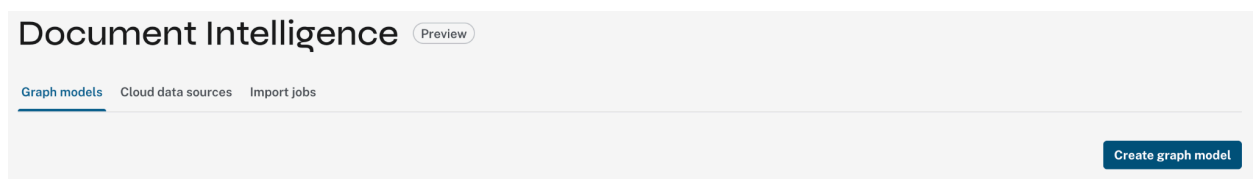
- Provide the data
- Generate a data model
- Refine the model
- Fix model errors
- Run the import

You need to have a running Aura instance available, currently Document Intelligence supports the Free, Professional, and Business Critical tiers. You will require your credentials for your instance as well.

Our hands-on exercise is focused on Aura today. For those wishing to extract documents into Neo4j without using Aura, we suggest the [neo4j-mcp-workspace-template](#). It is a great place to get started. It is a python based approach to document extraction using mcp servers.

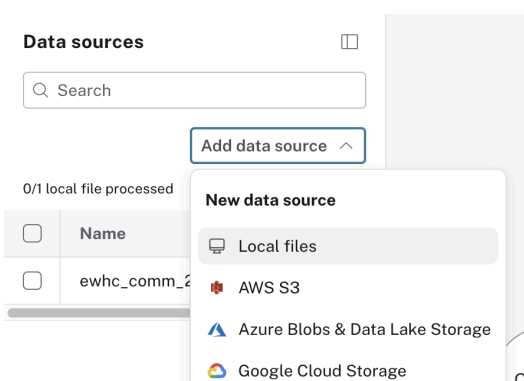
We begin by selecting 'Create Graph Model'

Create Graph Model

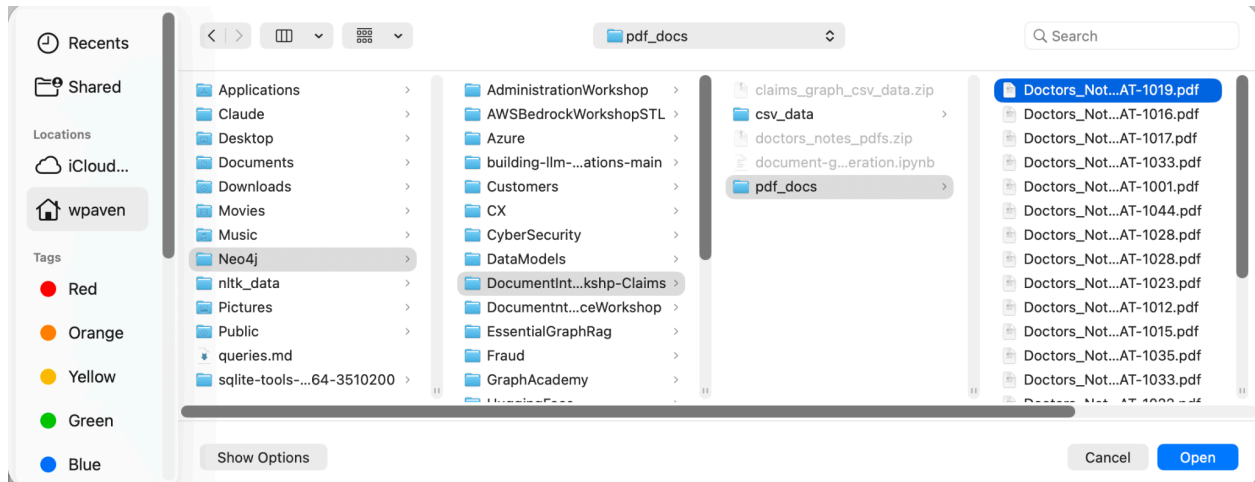


Add Data Source

Select 'Local Files' from the menu.



Select Local Files



Select 1 of the files from the data directory you downloaded from the workshop github repo.

Note: I used **Doctors_Note_CLM-10001_PAT-1019.pdf** to model, then added others, then imported all. Just select one file. The next screen may look different for you.

Generate Model

Data sources 

Add data source 

0/20 local files processed

☐	Name	Status
☐	Doctors_Note_CLM-1000 1_PAT-1019	 Ready
☐	Doctors_Note_CLM-1000 2_PAT-1016	 Ready
☐	Doctors_Note_CLM-1000 3_PAT-1017	 Ready
☐	Doctors_Note_CLM-1000 4_PAT-1033	 Ready
☐	Doctors_Note_CLM-1000 5_PAT-1001	 Ready
☐	Doctors_Note_CLM-1000 6_PAT-1044	 Ready

Click “Generate Model”.

Prompt to generate model

You’ll be prompted to provide context.

Add the following text to the Context form:

This data is being examined for claims processing. It is doctor’s notes where we want to focus on diagnoses and procedures.

Add context for model generation



Describe your data domain and what the generated graph model should focus on.

Context

This data is being examined for claims processing. It is doctor's notes where we want to focus on diagnoses and procedures.

Required to generate a more relevant model. 124/2000

Cancel

Generate model

Click 'Generate Model'

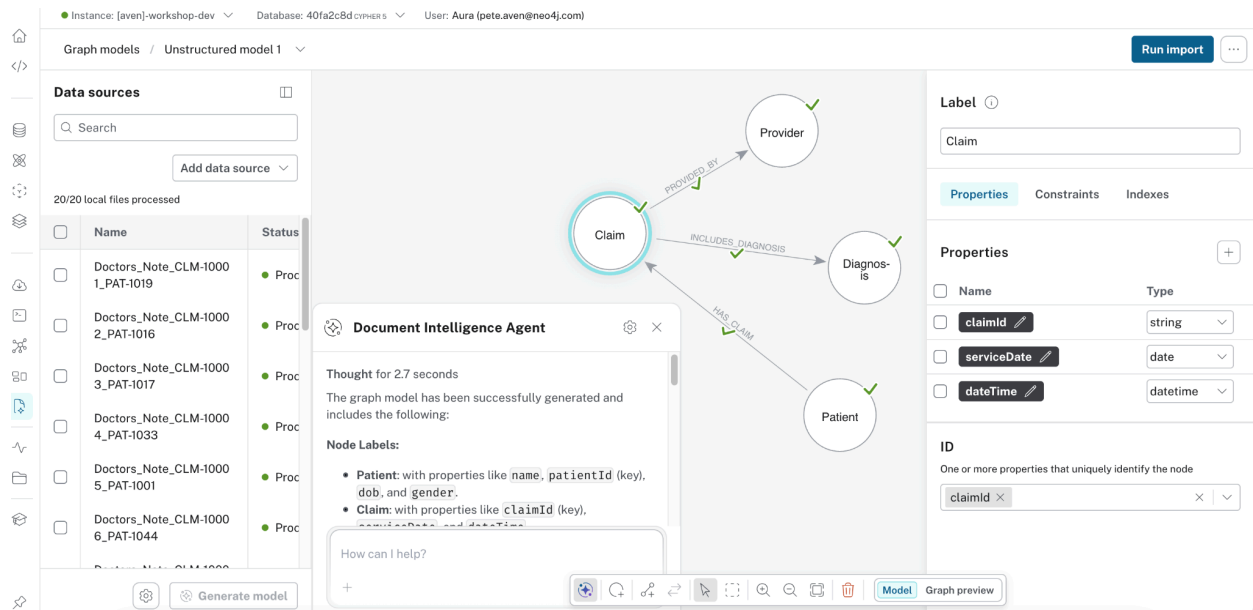


Generating model...

We're turning your sources into a graph model...

Cancel

Model Generated



A model is previewed along with reasoning provided by the Document Intelligence Agent (DIA). From here you can manually edit the model, or refine using the DIA. When I ran this, procedures and their codes were not identified, so I prompted the DIA to rectify this.

Refine Model

Document Intelligence Agent

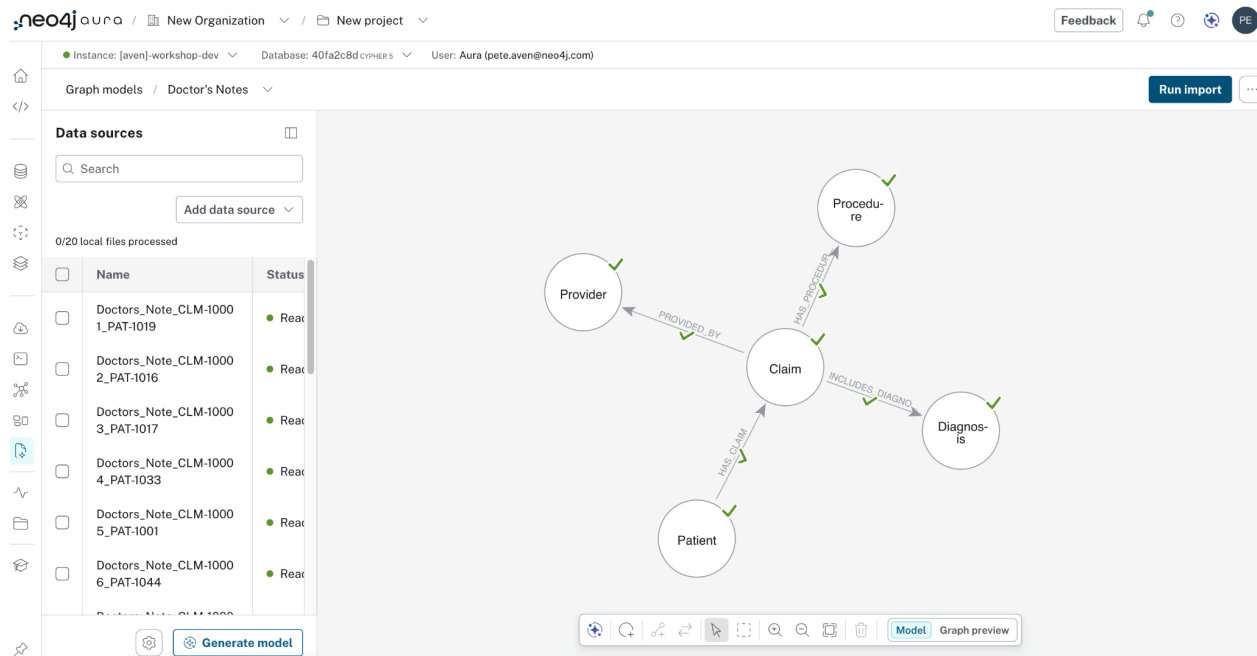
Here are some suggested next steps:

- 1 Generate a Graph Preview:** You can generate a preview to visualize how your document data would look in this graph structure. This will help you confirm if the schema accurately represents your data.
- 2 Validate the Schema:** It's always a good idea to validate the schema to check for any potential

Let's refine the schema. The claim should have a Procedure as well that includes a code and description.

Updated Model

The updated model now connects Procedure nodes to the Claims as well.



2. Load Data

To load our document into the model, we select 'Run Import' in the upper right hand corner.

Run import

Run import

×

From data source

Data source name

 Local files

To aura instance

Instance

● [aven]-workshop-dev

ID: 40fa2c8d ▾

Database User

40fa2c8d

Password

.....

Currently, only importing into empty databases is supported.

Please ensure that your database is empty, and that it does not contain any constraints.

Please refer to the [documentation](#) ↗ for information on dropping constraints.

Cancel

Run

Enter your credentials and select 'Run'

Run import

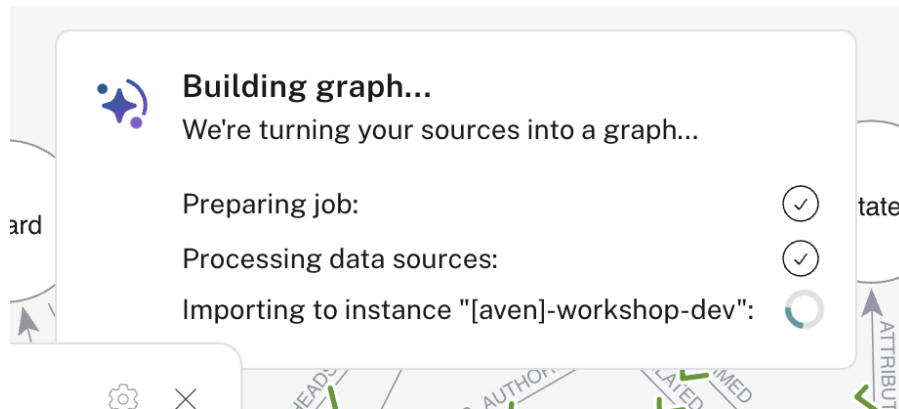
×

✓ Uploading files

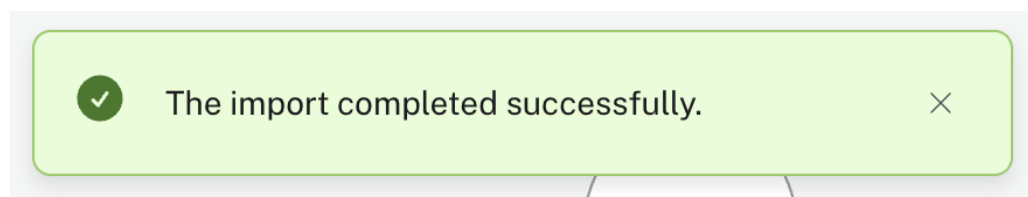
✓ Creating data source

○ Starting import

We are notified the process is running and that the graph is being built.



We are notified the process is running and that the graph is being built. We are notified when the process is complete.



Successful Import from DIA

The knowledge graph import job has completed successfully! We are provided statistics of what has been created and if there were any errors.

Note: Your statistics will differ as you've only uploaded one document and your model may be slightly different as well. We'll fix this later.

Here are the statistics from the import:

Status: Completed
Total Nodes Created: 104
Total Relationships Created: 200
Nodes per Label:
 Claim: 20
 Diagnosis: 8
 Patient: 18
 Procedure: 10
 Provider: 8
 __Chunk__: 20

__Document__: 20

Relationships per Type:

HAS_CLAIM: 20

HAS_PROCEDURE: 20

INCLUDES_DIAGNOSIS: 20

PROVIDED_BY: 20

__CHUNK_TO_DOCUMENT__: 20

__NODE_TO_CHUNK__: 100

Input Files Processed: 20 (out of 20 total, 0 failed)

Total Size of Input Files: 62,645 bytes

Now that your data has been successfully imported into the knowledge graph, here are some suggested next steps:

Query the Knowledge Graph: You can now ask questions about your data in natural language. For example, "Show me all claims for patient PAT-1019," or "What procedures were performed by Dr. Sarah Jenkins?"

Explore the Data: Use the graph visualization tools to explore the connections and patterns within your imported data.

Refine the Schema (if needed): If, after exploring the data, you identify any areas where the schema could be further optimized or expanded, we can make those adjustments.

Share Your Model: You can download and share your graph model with others.

What would you like to do next?

3. Query

Navigate to the 'Query' pane by selecting it from the left hand menu. Click on a chunk to query chunks. Double click on one of the chunks to expand its relationships. Select the nodes to see their properties.

neo4j aura / New Organization / New project

Instance: [aven]-workshop-dev Database: 40fa2c8d cypher5 User: Aura (pete.aven@neo4j.com)

Stream WHERE example DB Schema +

\$ Start typing or generate with AI (*I)

40fa2c8d\$ MATCH (n: `\_\_Document\_\_`) RETURN n LIMIT 25;

Graph Table Raw

Search graph...

Database information

Nodes (104)

- __Chunk__ __Document__
- __Entity__ Claim Diagnosis
- Patient Procedure Provider

Relationships (200)

- __CHUNK_TO_DOCUMENT__
- __NODE_TO_CHUNK__
- HAS_CLAIM HAS_PROCEDURE
- INCLUDES_DIAGNOSIS
- PROVIDED_BY

Property keys

- __id__ claimId code
- createdAt dateTime
- description dob
- document_type embedding
- file_identifier file_size
- from_label gender index
- name npi path patientId

Node details

__Entity__ Claim

Customize Style

Color:

Size:

Caption:

__id__ claimId dateTime serviceDate

<id> <type>

Styling the Results

Pro tip: If you select any of the gray in the canvas area with the nodes, you'll be presented with node details for all nodes in the display on the right. You can then select a node to customize that node type's color, style, and captions.

If you select the update arrow in the upper right corner, you'll see the option to update the styling priority. Click this up-down arrow to navigate to the prioritization screen.

Results overview

Update styling priority ↕

- __Chunk__ (20)
- __Document__ (1)
- __Entity__ (5) * (26)
- Claim (1) Diagnosis (1)
- Patient (1) Procedure (1)
- Provider (1)

Relationships (6)

- __CHUNK_TO_DOCUME...
- __NODE_TO_CHUNK__ (5)
- * (6)

You'll notice that `__Entity__` and `__Chunk__` nodes have multiple labels. When this occurs, everything in the query results could appear to be one color. By dragging them lower in the priority, we get a more clear visualization of the types of nodes displayed in our query results.

Results overview

Nodes (26)

↑↓

Provider

Procedure

Diagnosis

Patient

__Document__

__Chunk__

Claim

__Entity__

Payer

Relationships (6)

__CHUNK_TO_DOCUME...

__NODE_TO_CHUNK__ (5)

* (6)

Indexes and Constraints

Type 'SHOW CONSTRAINTS' to view the CONSTRAINTS created as part of ingesting the document model.

40fa2c8d\$ SHOW CONSTRAINTS

Run

×

TableRaw

Search table...

Graph chip

Download

Refresh

id	name	type	entityType	labelsOrTypes	properties	ownedIndex	propertyType
15	"Claim_key_claimId"	"NODE_KEY"	"NODE"	["Claim"]	["claimId"]	"Claim_key_claimId"	null
7	"Diagnosis_key_code"	"NODE_KEY"	"NODE"	["Diagnosis"]	["code"]	"Diagnosis_key_code"	null
9	"Patient_key_patientId"	"NODE_KEY"	"NODE"	["Patient"]	["patientId"]	"Patient_key_patientId"	null
5	"Procedure_key_code"	"NODE_KEY"	"NODE"	["Procedure"]	["code"]	"Procedure_key_code"	null
3	"Provider_key_npi"	"NODE_KEY"	"NODE"	["Provider"]	["npi"]	"Provider_key_npi"	null
13	"__Chunk__key"	"NODE_KEY"	"NODE"	["__Chunk__"]	["__id__"]	"__Chunk__key"	null
11	"__Document__key"	"NODE_KEY"	"NODE"	["__Document__"]	["__id__"]	"__Document__key"	null

Started streaming 7 records after 11 ms and completed after 11 ms.

Type 'SHOW INDEXES' to view the INDEXES. Of importance here, not the 'VECTOR' index. Embeddings for our extracted text properties were automatically created for us. Note that Document Intelligence uses the **gemini-embedding-001** model to create the embeddings. This is important as we'll need to use this same model for querying.

id	name	state	population	type	entityType	labelsOrType	properties	indexProv	owningCon	lastRead	readCount
	__key"					__"]		1.0"	__key"	11T15:30:48.631000000Z	
7 10	"__Docume nt__key"	"ONLINE"	100.0	"RANGE"	"NODE"	["__Docume nt__"]	["__id__"]	"range- 1.0"	"__Docume nt__key"	2026-08-11T15:30:45.831000000Z	60
8 1	"index_1b 9dcc97"	"ONLINE"	100.0	"LOOKUP"	"RELATION SHIP"	null	null	"token- lookup- 1.0"	null	null	0
9 0	"index_46 0996c0"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token- lookup- 1.0"	null	2026-08-11T16:57:28.836000000Z	13
10 16	"vector_i ndex__Ch unk__emb edding"	"ONLINE"	100.0	"VECTOR"	"NODE"	["__Chunk_ __"]	["__embeddin g"]	"vector- 2026.07"	null	null	0

Restore the Database from Backup

At this point we all have similar, but likely different in some ways from each other, document models. We've also only imported data from one document. So we all start with the same foundation, we'll restore our database using the dump file in the [workshop github repository](#).

We will use the **claims_notes.backup** file. This file includes the Document Model illustrated in this workshop along with data extracted into the model for 20 documents representing Doctor's notes.

Note: When you restore a database, it will overwrite the entire database with the contents of the backup file. So if you have additional data in your database you wish to save, be sure to take a backup first.

Select Your Database

Select 'Instances' from the menu pane on the left.

 Get started

 Developer hub

Data services

 Instances

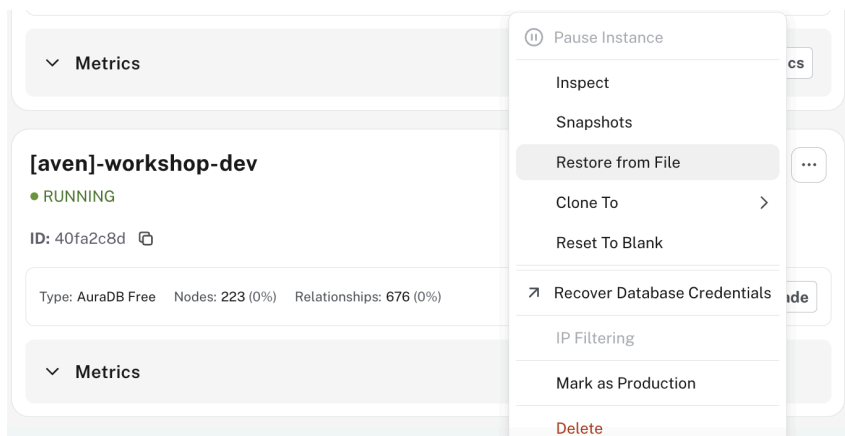
 Graph Analytics

 GraphQL Data APIs

 Agents

New

Click the ellipsis ‘...’ in the upper right corner for your instance to display the menu items.



Select 'Restore From File'

[aven]-workshop-dev ● RUNNING

Overview Snapshots **Restore from File** SSO membership



Drag & Drop your Neo4j **.dump** or **.backup** or **.tar** file here (4GB or less)

or [Browse](#)

Want to upload a dump or backup larger than 4GB? [Click here](#) to view the Neo4j Admin 'upload' command in the Aura Docs.

The graph you upload should be smaller than 200,000 nodes and 400,000 relationships.

Drop the **claims_notes.backup** file into the designated area or select it using 'Browse'.

Are you sure?

All existing data in your database will be replaced with claims_notes.backup (584.12 KB)

Cancel

Upload

You will be prompted to confirm you want to restore from backup as this will overwrite the contents of the database. Click 'Upload'

[aven]-workshop-dev ● LOADING

Overview Snapshots **Restore from File** SSO membership

Loading in progress
Import feature will be available again once current data loading completes

You will then see that Loading is in Progress. The database will be unavailable during this time.

[aven]-workshop-dev

● LOADING

ID: 40fa2c8d 

Once complete, the database will show as available in a 'Running' state.

[aven]-workshop-dev

● RUNNING

ID: 40fa2c8d 

Navigate to the Query pane. You will see there are 104 nodes and 200 relationships. If you click on a `__Document__` node in the query panel, you will find 20 are returned.

You can validate the CONSTRAINTS and INDEXES have been created as you wish. Once you're done validating, we are now ready to load our structured data.

LOAD STRUCTURED DATA

Now that we've loaded our documents, we're going to load our structured data. Loading document chunks first is not required. Many times you'll be starting with structured data in your knowledge graph, and want to append the unstructured data later as your knowledge graph continues to grow. For the purposes of this workshop, we lead with loading documents first to highlight Document Intelligence. But the order will depend on your use case requirements.

1. Uniqueness Constraints (Additional)

Creating unique constraints first ensures high-performance indexing during the **MERGE** operations. Many were already created for us when we ran document intelligence. We just need to add a constraint for our Payer nodes. Payers will be added to the graph separately as there was no Payer info in the doctor's notes.

Run the following in the Query pane in Aura.

Cypher

None

```
CREATE CONSTRAINT Payer_key_code IF NOT EXISTS FOR (p:Payer)
  REQUIRE p.payerId IS NODE KEY
```

You can run SHOW CONSTRAINTS to validate the constraint is created.

We will be adding additional nodes, additional relationships, and adding properties to some relationships to enrich the graph. Some relationships already exist. These will be updated accordingly. Others will be created as needed to connect new nodes that we'll be adding that come from our structured sources.

2. Load Nodes

[LOAD CSV](#) is used to import data from CSV files into a Neo4j database. There are [Connectors](#) and [Drivers](#) as well. For this workshop we use LOAD CSV in the interest of keeping things simple.

For each of the following, run the Cypher in the Query pane of the Aura UI.

Before running, note the number of nodes for the given node type before loading, and after. As we've added constraints, nodes for existing entities will be merged and new ones will be created.

```
MATCH (n:Patient) RETURN count(n)
```

The above statement returns **18** prior to loading structured data, and **50** after we run our **LOAD CSV** statement..

Also take a look at the properties for nodes prior to loading. For Patients, we'll see new properties appended to existing nodes that come from our CSV files.

Node details

__Entity__ Patient

Key	Value
__id__	"ebf3a25a-7d70-45d9-92ef-7e7246f281d0:patient_PAT-1012"
<id>	4:5a2d5546-668a-4cd2:34
dob	"1974-12-09"
gender	"Female"
name	"Patricia Gonzalez"
patientId	"PAT-1012"

A Patient **BEFORE** loading structured data

Node details

__Entity__ Patient

Key	Value
__id__	"ebf3a25a-7d70-45d9-92ef-7e7246f281d0:patient_PAT-1012"
<id>	4:5a2d5546-668a-4cd2:34
birthDate	1974-12-09
dob	"1974-12-09"
firstName	"Patricia"
gender	"Female"
lastName	"Gonzalez"
name	"Patricia Gonzalez"
patientId	"PAT-1012"
zipCode	"98101"

A Patient **AFTER** loading structured data

We load our nodes first, and then load our relationships. Run the following in the Query pane in the Aura UI.

A. Load Patients (**Patient.csv**)

Adds 32 Patient nodes, Merges 18

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Patient.csv' AS line
MERGE (p:Patient {patientId: line.patient_id})
SET p.firstName = line.first_name,
p.lastName = line.last_name,
p.gender = line.gender,
p.birthDate = date(line.birth_date),
p.zipCode = line.zip_code;
```

B. Load Payers (**Payer.csv**)

Merges 5 Payer nodes (Creates, these are new)

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Payer.csv' AS line
MERGE (p:Payer {payerId: line.payer_id})
SET p.name = line.name,
p.planType = line.plan_type;
```

C. Load Providers (**Provider.csv**)

Merges 8 Provider nodes

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Provider.csv' AS line
MERGE (pr:Provider {npi: line.npi})
SET pr.name = line.name,
pr.specialty = line.specialty,
pr.providerType = line.provider_type;
```

D. Load Diagnoses (**Diagnosis.csv**)

Adds 2 Diagnosis nodes, Merges 8 nodes

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Diagnosis.csv' AS line
MERGE (d:Diagnosis {code: line.code})
SET d.description = line.description;
```

E. Load Procedures (**Procedure.csv**)

Merges 10 Procedure nodes

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Procedure.csv' AS line
MERGE (p:Procedure {code: line.code})
SET p.description = line.description;
```

F. Load Claims (**Claim.csv**)

Look at before and after for a Claim node in the Query Pane.

Adds 80 Claim nodes, Merges 20

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Claim.csv' AS line
MERGE (c:Claim {claimId: line.claim_id})
SET c.serviceDate = date(line.service_date),
c.claimType = line.claim_type,
c.totalBilledAmount = toFloat(line.total_billed_amount),
c.totalPaidAmount = toFloat(line.total_paid_amount),
c.status = line.status;
```

3. Create Relationships

Relationships at 200 to start. The following adds 260,
Claims will now be connected with Payer as well.

A. Link Claims to Patient, Payer, and Provider (Claim.csv)\

Creates 260 relationships

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Claim.csv' AS line
MATCH (c:Claim {claimId: line.claim_id})
MATCH (pat:Patient {patientId: line.patient_id})
MATCH (pay:Payer {payerId: line.payer_id})
MATCH (pr:Provider {npi: line.provider_npi})

MERGE (pat)-[:HAS_CLAIM]->(c)
MERGE (c)-[:BILLED_TO]->(pay)
MERGE (c)-[:PROVIDED_BY]->(pr);
```

B. Link Claims to Diagnoses (Claim_Diagnosis.csv)

Now enriching relationship with sequence number
Creates 112 relationships

Cypher

```
LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Claim_Diagnosis.csv' AS
line
MATCH (c:Claim {claimId: line.claim_id})
MATCH (d:Diagnosis {code: line.diagnosis_code})

MERGE (c)-[r:INCLUDES_DIAGNOSIS]->(d)
SET r.type = line.type,
r.sequence = toInteger(line.sequence);
```

C. Link Claims to Procedures (Claim_Procedure.csv)

Creates 104 relationships
Sets properties on relationships.

Cypher

LOAD CSV WITH HEADERS FROM

```
'https://raw.githubusercontent.com/neo4j-field/field-document-intelligence-workshop-claims/refs/heads/main/data/csv/Claim_Procedure.csv' AS line
```

```
MATCH (c:Claim {claimId: line.claim_id})
```

```
MATCH (p:Procedure {code: line.procedure_code})
```

```
MERGE (c)-[r:HAS_PROCEDURE]->(p)
```

```
SET r.billed_amount = toFloat(line.billed_amount),
```

```
r.allowed_amount = toFloat(line.allowed_amount),
```

```
r.units = toInteger(line.units);
```

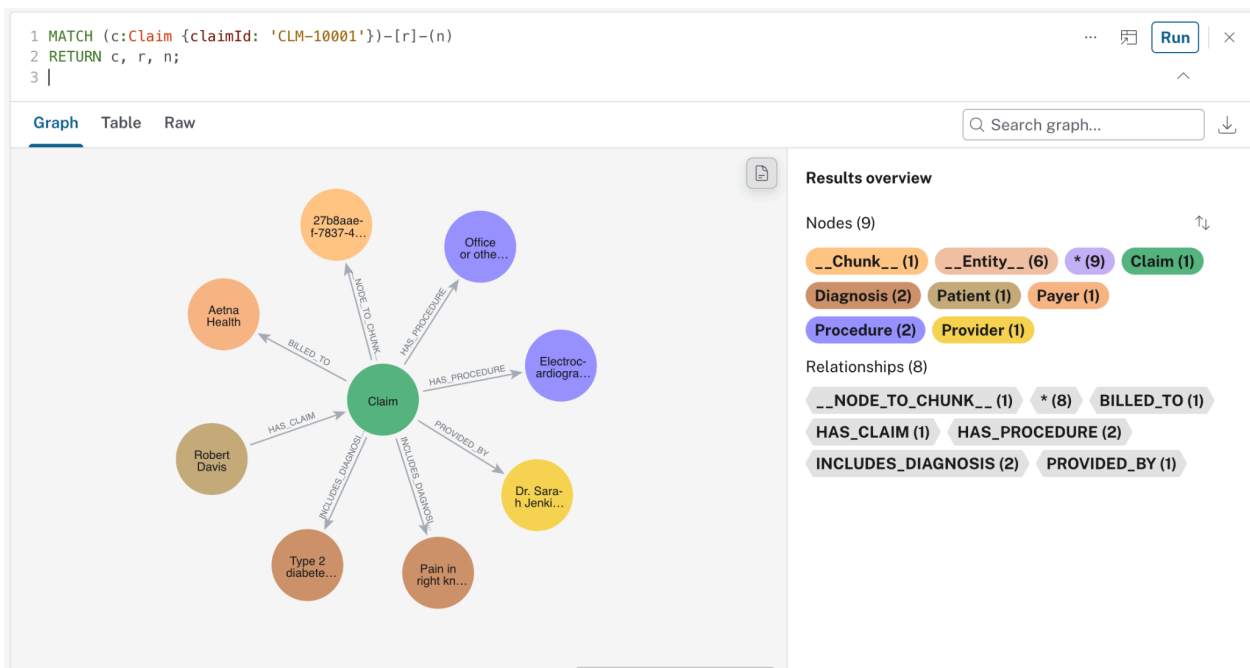
4. Quick Verification Query

After running the scripts, execute this Cypher query to verify that the graph was constructed properly:

Cypher

None

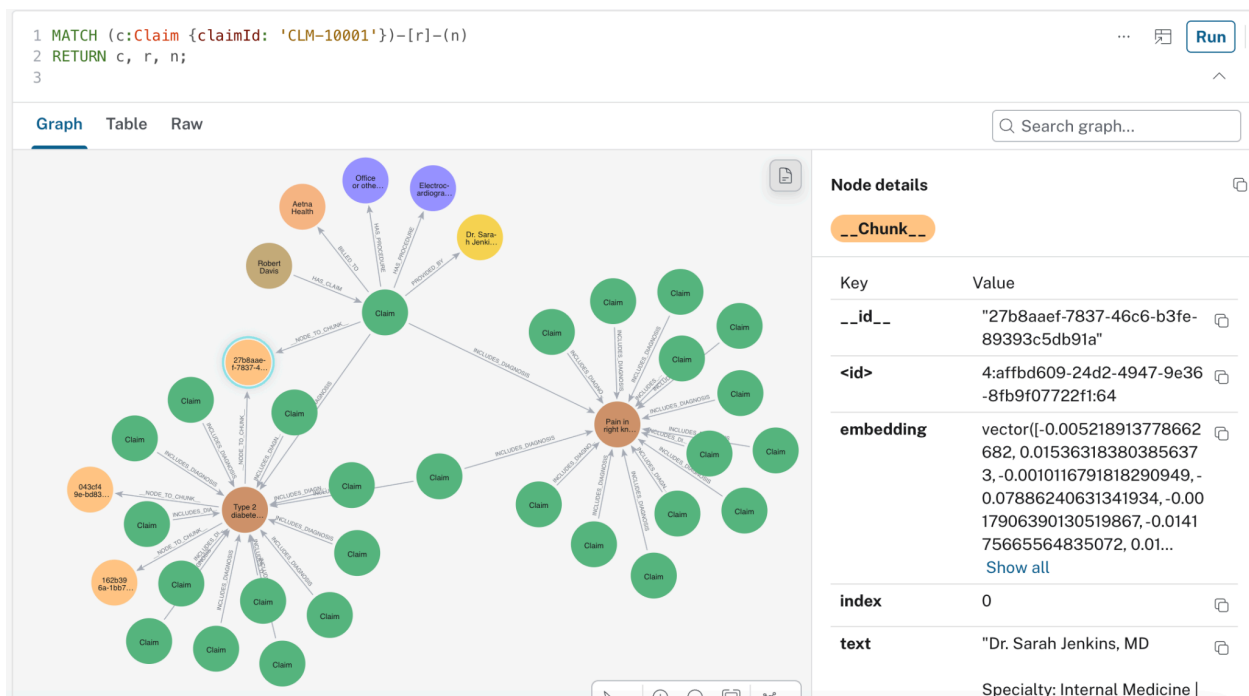
```
MATCH (c:Claim {claimId: 'CLM-10001'})-[r]-(n)
RETURN c, r, n;
```



Double click each Diagnosis for this Claim to expand its relationships.

Clicking on each INCLUDES_DIAGNOSIS relationship, you can find the sequence in which the Diagnosis was provided by viewing the sequence property in the right hand pane off Aura.

You'll also notice one connects to the Doctor's notes, the other does not. We don't have Doctor's notes for all Claims in our example. And this is what it looks like in business as well. We may be waiting for those notes to become available so we can later extract, ingest, and append them to the graph.



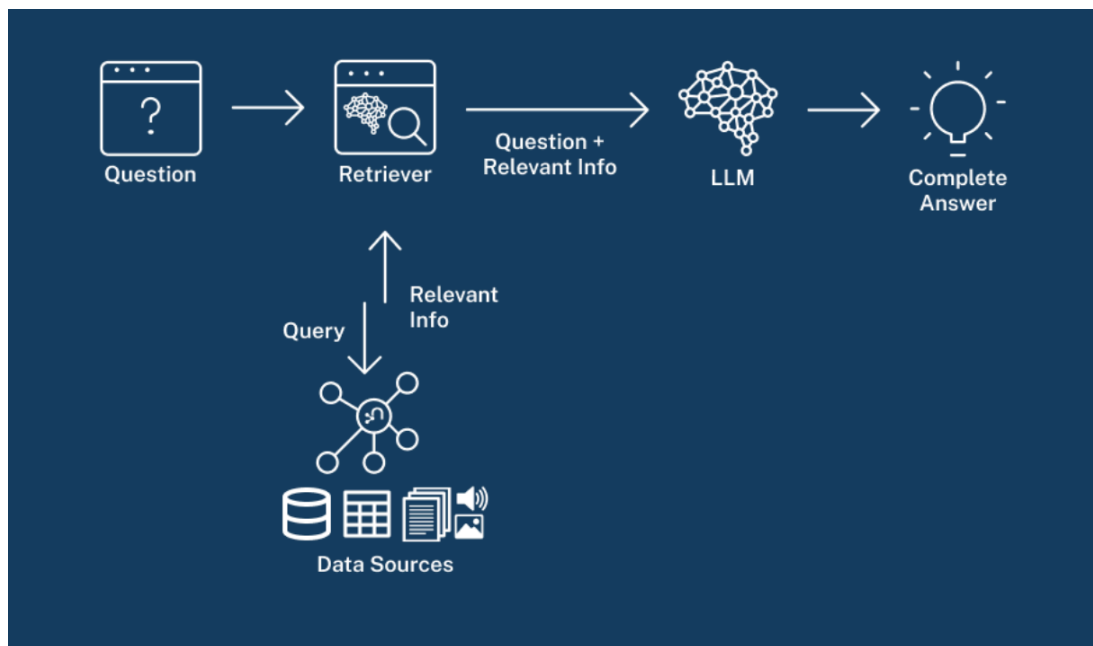
NOTE: Backup Available - If you were not able to run LOAD CSV for some reason. Or have some other issue you are facing. You can restore your database using [dump/claims_csv_plus_notes.backup](#) in the workshop github repo. This backup contains the complete database of parsed unstructured documents and the structured csv files we've loaded up to this point so you can start the next section with a solid foundation.

Queries and GraphRAG

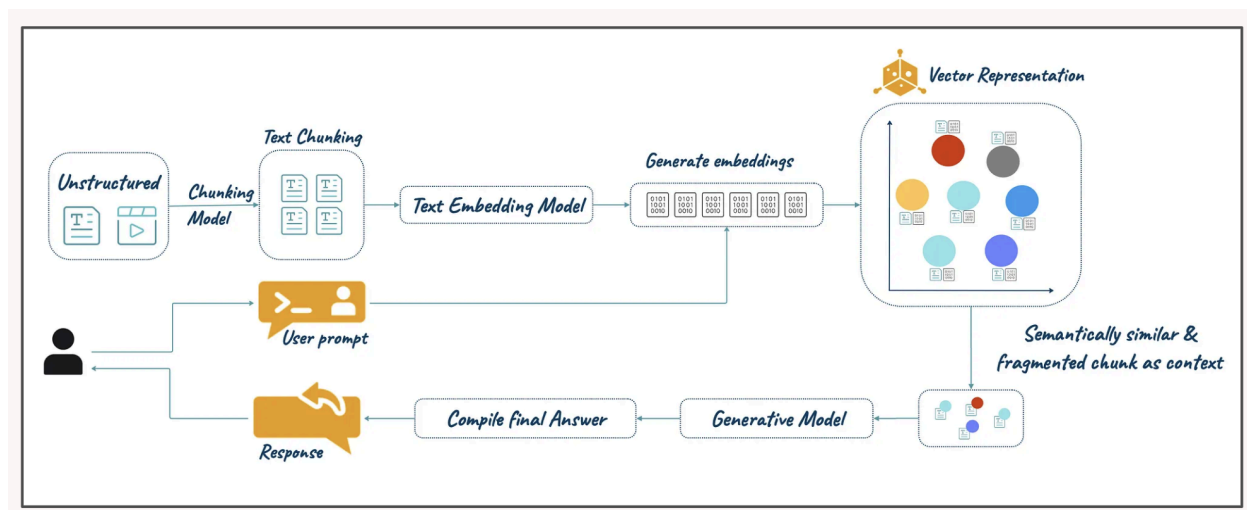
GraphRAG is Retrieval Augmented Generation (RAG) using Knowledge Graphs. The benefit of this approach is context. We can provide more relevant info to the LLM and this helps reduce

hallucinations and increase the accuracy of our LLM responses. We also send less, more contextually relevant data to the LLM, thus reducing token spend as compared to traditional RAG approaches (without the benefit of the knowledge graph)

The GraphRAG pattern from a high level follows. The user provides a question, a retriever queries for relevant information and passes these results along with the original question to the LLM, which provides our complete response.



But this misses a bit of detail. Digging into it deeper, the pattern looks more like this.



A user provides a question in the form of natural language (“Can you tell me about Claims for Thomas Martin?”). The first step is we need to take that text, and generate the vector embedding for it. With this embedding, we can then do a semantically similar search for nodes that match the criteria using the vector index to match the query embeddings with the embeddings we’ve persisted in the nodes. With these nodes, we can then pass their data along with the original question to the LLM, which will use its knowledge, combine it with the knowledge we’ve provided, and return a complete answer.

This workshop is intended to be completed entirely in Aura. But we need to generate embeddings to be able to ask questions and retrieve information from the vector index. So for an example in Aura, we’ve provided the embedding for a query in the Appendix. That said, that’s completely unwieldy and not how we’d generally perform these types of queries, so we have provided a Jupyter notebook to illustrate examples you can run on your own outside of the workshop. We’ll be referring to that in this section to provide useful examples.

Install dependencies

```
pip install google-genai neo4j
```

Generate embedding to pass as parameter. Searching for ‘chest pain’

```
from google import genai
from google.genai import types#1. Establish client

client = genai.Client(api_key=GEMINI_API_KEY)

# 2. Call models.embed_content
result = client.models.embed_content(
    model="gemini-embedding-001",
    contents="chest pain",
    config=types.EmbedContentConfig(
        task_type="RETRIEVAL_QUERY"
    )
)

# 3. Access the embedding values
print(result.embeddings[0].values)
```

Python

Document Intelligence currently uses the **gemini-embedding-001** model. This model has a 3,072 dimensional vector output size.

You can use your own models to generate text embeddings outside of Document Intelligence. You can use these to store in Neo4j as text embeddings for your text fields in nodes as well. But the same model used to create text embeddings is the one required to query those embeddings. The Google API is also unique in that the API requires you specify if you are

creating embeddings for a document (**RETRIEVAL_DOCUMENT**) or for a query (**RETRIEVAL_QUERY**)

You can sign up for [Google AI Studio](#) to get a free account. This account will allow you to use it to create embeddings for the search phrases we'll want to use to query the database. Replace the **GEMINI_API_KEY** with your API key that will be provided when you sign up for your Google AI Studio account.

Likewise, when using **document-query.ipynb** be sure to replace the **NEO4J*** variables with the values provided for your free instance of Aura. When you created your free instance, there was an option to download the credentials file. All required information is in that file.

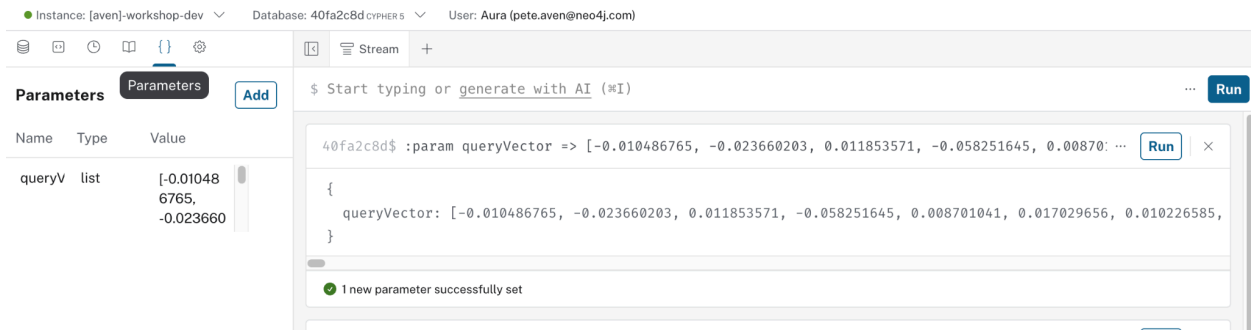
You can run the notebook cell above to generate the embedding for 'chest pain'. If you're not able to run this, please see the appendix for the vector embedding for 'chest pain'. Copy it into the Query U using the following example (truncated due to length of vector embedding). Click run to set the parameter in memory.

Add param to UI

:param queryVector => [{vector embedding from **appendix** or your notebook output }]

The screenshot displays the Neo4j Aura Query UI. On the left, the 'Database information' panel shows 'Nodes (223)' with categories like __Chunk__, __Document__, __Entity__, Claim, Diagnosis, Patient, Payer, Procedure, and Provider. It also shows 'Relationships (676)' with types like __CHUNK_TO_DOCUMENT__, __NODE_TO_CHUNK__, BILL_TO, HAS_CLAIM, HAS_PROCEDURE, INCLUDES_DIAGNOSIS, and PROVIDED_BY. The main area shows a query execution result for the parameter 'queryVector', displaying a long list of floating-point numbers representing a vector embedding. A 'Run' button is visible in the top right corner of the query editor.

We can now view it in the parameters tab of the Query UI.



Now that we have set the parameter, we can use it in our Vector Search query using the vector index. Copy each of the following and run them in the Query UI individually..

Now can query using built-in function or newer CYPHER 25 SEARCH clause (GQL aligned)

```
CALL db.index.vector.queryNodes("vector_index__Chunk__embedding",
20, $queryVector)
YIELD node, score
RETURN node.text AS text, score
LIMIT 5
```

```
CYPHER 25
WITH [x IN $queryVector | toFloat(x)] AS floatVector
MATCH (chunk:__Chunk__)
SEARCH chunk IN (
VECTOR INDEX vector_index__Chunk__embedding FOR floatVector
LIMIT 5
) SCORE AS score
RETURN chunk.text, score
```

The above are examples of a **Vector Query**. But what if we wanted to search for Chunks, but only those Chunks with relationships to Claims that had not been Denied. Then we would want to use a **Vector Cypher Query**.

The following is still searching for 'chest pain'. You can change the search by providing a new vector embedding or using the python notebook.

```
CYPHER 25
MATCH (chunk:__Chunk__)
SEARCH chunk IN (
VECTOR INDEX vector_index__Chunk__embedding FOR $queryVector
```

```

LIMIT 5
) SCORE AS score
MATCH (claim:Claim)-[:__NODE_TO_CHUNK__]->(chunk)
WHERE claim.status <> 'Denied'
RETURN score, claim.claimId AS claimId, claim.status AS status

```

These examples have been provided to:

- Demonstrate a component of GraphRAG in action
 - We just performed our first simple query for GraphRAG. More to come though...
 - Demonstrate a Vector Query
 - Demonstrate a Vector Cypher Query
- Remind you Neo4j has built-in functions
- Recommend you use CYPHER 25 as this is aligned with GQL

But normally, we wouldn't copy a huge vector into the Query UI like this. This may be useful for getting familiar with Aura, but we'll generally want to use python or some other language when working with embeddings and information stored in Neo4j. This will allow us to be more dynamic with our interactions...

Notebook Examples

Python

The following illustrate the same way to achieve the above in python.

These are examples of a **Vector Query**.

```

# --- 1. GENERATE GEMINI QUERY EMBEDDING ---
ai_client = genai.Client(api_key=GEMINI_API_KEY)

# --- 2. Retrieve embeddings for search query 'diabetes'
embedding_response = ai_client.models.embed_content(
    model="gemini-embedding-001",
    contents="chest pain",
    config=types.EmbedContentConfig(
        task_type="RETRIEVAL_QUERY"
    )
)

queryVector = [float(val) for val in embedding_response.embeddings[0].values]

# --- 3. EXECUTE NEO4J VECTOR SEARCH ---

```

```

# Built in Function
cypher_query = """
CALL db.index.vector.queryNodes($index_name, $k, $queryVector)
YIELD node, score
RETURN node.text AS text, score
LIMIT $k
"""

# CYPHER 25 SEARCH Clause - GQL Aligned
cypher25_query = """
CYPHER 25
MATCH (chunk:__Chunk__)
SEARCH chunk IN (
  VECTOR INDEX vector_index__Chunk__embedding FOR $queryVector
  LIMIT $k
) SCORE AS score
RETURN chunk.text AS text, score
"""

with GraphDatabase.driver(NEO4J_URI, auth=NEO4J_AUTH) as driver:
    records, summary, keys = driver.execute_query(
        cypher25_query,
        index_name=NEO4J_VECTOR_INDEX,
        k=TOP_K,
        queryVector=queryVector,
        database_=NEO4J_DATABASE
    )

# --- 4. PRINT RESULTS ---
for record in records:
    print(f"Score: {record['score']:.4f}")
    content_str = str({record['text']})
    preview = content_str[:200] + '...' if len(content_str) > 200 else content_str
    print(f"Text: {preview}")
    #print(f"Score: {record['score']:.4f} | Text: {record['text']}")

```

The output of running this:

Score: 0.8456

```

Text: {'Dr. David Miller, MD \n\nSpecialty: Gastroenterology | NPI: 1841234506 \n\n123
Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...
Score: 0.8397
Text: {'Dr. David Miller, MD \n\nSpecialty: Gastroenterology | NPI: 1841234506 \n\n123
Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...
Score: 0.8388
Text: {'Dr. Sarah Jenkins, MD \n\nSpecialty: Internal Medicine | NPI: 1841234501
\n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...
Score: 0.8382
Text: {'Metro General Hospital \n\nSpecialty: General Hospital | NPI: 1841234507
\n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...
Score: 0.8378
Text: {'Dr. Sarah Jenkins, MD \n\nSpecialty: Internal Medicine | NPI: 1841234501
\n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...

```

The example of a **Vector Cypher Query** has been updated to search for 'diabetes' and for Claims that have been Denied..

```

#Query Embeddings and Relationships
# --- 1. Retrieve embeddings for search query 'diabetes'
embedding_response = ai_client.models.embed_content(
    model="gemini-embedding-001",
    contents="diabetes",
    config=types.EmbedContentConfig(
        task_type="RETRIEVAL_QUERY"
    )
)

queryVector = [float(val) for val in embedding_response.embeddings[0].values]

# --- 2. Query for embeddings where Chunks related to Claims that have been Denied
# Query includes chunks and their relationships to Claims nodes
cypher25_query = """
CYPHER 25
MATCH (chunk:__Chunk__)
SEARCH chunk IN (
    VECTOR INDEX vector_index__Chunk__embedding FOR $queryVector

```

```

LIMIT $k
) SCORE AS score
MATCH (claim:Claim)-[:__NODE_TO_CHUNK__]->(chunk)
WHERE claim.status = 'Denied'
RETURN score, claim.claimId AS claimId, claim.status AS status
"""

with GraphDatabase.driver(NEO4J_URI, auth=NEO4J_AUTH) as driver:
    records, summary, keys = driver.execute_query(
        cypher25_query,
        index_name=NEO4J_VECTOR_INDEX,
        k=TOP_K,
        queryVector=queryVector,
        database_=NEO4J_DATABASE
    )

# --- 3. Print Results
for record in records:

```

Neo4j GraphRAG for Python

[The Ne4j GraphRAG for Python](#)) is an official, Neo4j supported library designed to bridge **graph databases** with **Generative AI**. It allows developers to build advanced Retrieval-Augmented Generation (RAG) applications by combining unstructured vector searches with the deeply structured context of a Neo4j knowledge graph. With it we can build a complete GraphRAG question-answering pipeline.

The following cel introduces the `VectorRetriever` for semantic search and connects it to an LLM to build a complete GraphRAG question-answering pipeline.

The retrieval pipeline works in three steps: the query is embedded, the vector index returns the most similar chunks, and those chunks are passed to the LLM as context for generating an answer.

VectorRetriever

To create our VectorRetriever pipeline, first we'll define a function to return an embedder for us.

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
# define an embedder function for our queries
def get_embedder(task_type: str = "retrieval_query"):
    """
    Returns an operational Gemini embedding object.
    Supported task_types: "retrieval_document", "retrieval_query",
"semantic_similarity"
    """
    # The SDK automatically checks for os.environ["GOOGLE_API_KEY"]
    return GoogleGenerativeAIEmbeddings(
        model="models/gemini-embedding-001",
        task_type=task_type
    )
```

Next, we'll initialize our retriever.

```
from neo4j_graphrag.retrievers import VectorRetriever
from neo4j_graphrag.generation import GraphRAG

#initialize a VectorRetriever

embedder = get_embedder() #note Google API requires parameter

driver = GraphDatabase.driver(
    NEO4J_URI,
    auth=(NEO4J_USER, NEO4J_PASSWORD),
    notifications_min_severity='OFF',
)
driver.verify_connectivity()

vector_retriever = VectorRetriever(
    driver=driver,
    index_name=NEO4J_VECTOR_INDEX,
    embedder=embedder,
    return_properties=['text']
)
```

```
print('Vector retriever initialized!')
```

Now we can test our retriever by passing it a natural language query for our database.

```
#test our retriever with a natural language query
query = "Which patients have diabetes?"
result = vector_retriever.search(query_text=query, top_k=5)

print(f'Query: "{query}"')
print(f'Results returned: {len(result.items)}\n')

for i, item in enumerate(result.items, 1):
    score = item.metadata.get('score', 0.0)
    content_preview = str(item.content)[:150]
    print(f'{i}. Score: {score:.4f}')
    print(f'    {content_preview}...\n')
```

And finally, we'll want to pass these results to an LLM to process and return a complete response for us. First thing we need to do is define a function to return that LLM. This model will be different from our text embedding model as it has broader capabilities than just creating and interpreting vector embeddings.

```
"""def get_llm(model_name: str = "gemini-2.0-flash") -> genai.Client:

    Initializes and returns a Google GenAI Client.
    Requires GEMINI_API_KEY to be set in the environment.

    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        raise ValueError("GEMINI_API_KEY environment variable is not set.")

    # The client automatically picks up the GEMINI_API_KEY variable,
    # but passing it explicitly prevents hidden runtime failures.
    client = genai.Client(api_key=api_key)
    return client

"""

def get_llm():
    # Instantiate and return the neo4j_graphrag GeminiLLM wrapper
    return GeminiLLM(
```

```

    model_name="gemini-3.6-flash",
    model_params={"temperature": 0.0},
    # api_key=os.getenv("GEMINI_API_KEY") # optional if set in env vars
)

```

Finally, we can bring everything together in a our VectorRetriever pipeline to

- Capture the user question
- Return embeddings for the question
- Retrieve the correct results from Neo4j
- Pass these results along with the original question to the LLM, which will generate our complete response for us.

```

from neo4j_graphrag.generation import GraphRAG

# What get_llm() safely instantiates for you under the hood:
from neo4j_graphrag.llm import GeminiLLM # For google-genai
# OR
#from neo4j_graphrag.llm import VertexAILLM # For vertexai

# Then it passes seamlessly into your workshop tasks:
llm = get_llm()
rag = GraphRAG(llm=llm, retriever=vector_retriever)

query = "What are risk codes associated with these patients?"
response = rag.search(query, retriever_config={'top_k': 5}, return_context=True)

print(f'Query: "{query}"')
print(f'Chunks retrieved: {len(response.retriever_result.items)}\n')
print('Answer:')
print(response.answer)

print('\n\n=== Retrieved Context ===')
for i, item in enumerate(response.retriever_result.items, 1):
    score = item.metadata.get('score', 0.0)
    content_str = str(item.content)
    preview = content_str[:200] + '...' if len(content_str) > 200 else content_str
    print(f'\n[{i}] Score: {score:.4f}')
    print(f'    {preview}')

```

The output:

Query: "What are risk codes associated with these patients?"

Chunks retrieved: 5

Answer:

Based on the provided clinical notes, the diagnosis/risk codes associated with these patients are:

```
* **R07.9** - Chest pain, unspecified
  * **Robert Garcia** (Patient ID: PAT-1022)
  * **William Williams** (Patient ID: PAT-1017)
  * **Linda Davis** (Patient ID: PAT-1035)

* **J06.9** - Acute upper respiratory infection, unspecified
  * **Patricia Wilson** (Patient ID: PAT-1041)

* **M54.5** - Low back pain
  * **Patricia Smith** (Patient ID: PAT-1001)
```

=== Retrieved Context ===

```
[1] Score: 0.8400
    {'text': 'Dr. David Miller, MD \n\nSpecialty: Gastroenterology | NPI:
1841234506 \n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555)
019-2834 \n\n\n+-----+-----...

[2] Score: 0.8395
    {'text': 'Dr. David Miller, MD \n\nSpecialty: Gastroenterology | NPI:
1841234506 \n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555)
019-2834 \n\n\n+-----+-----...

[3] Score: 0.8394
    {'text': 'Dr. David Miller, MD \n\nSpecialty: Gastroenterology | NPI:
1841234506 \n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555)
019-2834 \n\n\n+-----+-----...

[4] Score: 0.8393
    {'text': 'Dr. Amanda Patel, MD \n\nSpecialty: Pulmonology | NPI: 1841234505
\n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555) 019-2834
\n\n\n+-----+-----...

[5] Score: 0.8389
    {'text': 'Dr. Sarah Jenkins, MD \n\nSpecialty: Internal Medicine | NPI:
1841234501 \n\n123 Healthcare Plaza, Suite 400, Medical City | Tel: (555)
019-2834 \n\n\n+-----+-----...
```

VectorCypherRetriever

The VectorCypher Retriever allows us to pass a Cypher query along with our vector query so we return chunks associated with the relationships and other nodes that are important to our query.

Let's define our Cypher Query

```
NEO4J_RETRIEVAL_QUERY = """
MATCH (patient:Patient)-[:HAS_CLAIM]->(claim:Claim)-[:__NODE_TO_CHUNK__]->(node)
WITH node, score, patient, claim
RETURN node.text AS text,
        score,
        { claimID: claim.claimId,
          claimStatus: claim.status,
          patientName: patient.name
        } AS metadata
"""
print('Retrieval query defined!')
```

A helper function to format our results and more importantly, we initialize our VectorCypher Retriever.

```
import neo4j
from neo4j_graphrag.retrievers import VectorCypherRetriever
from neo4j_graphrag.types import RetrieverResultItem

def format_record(record: neo4j.Record) -> RetrieverResultItem:
    """Separate chunk text (content for LLM) from structured graph metadata."""
    metadata = record.get("metadata") or {}
    metadata["score"] = record.get("score")
    return RetrieverResultItem(
        content=record.get("text", ""),
        metadata=metadata,
    )

#updated retriever to use query and formatter
vector_cypher_retriever = VectorCypherRetriever(
    driver=driver,
    index_name=NEO4J_VECTOR_INDEX,
    embedder=embedder,
    retrieval_query=NEO4J_RETRIEVAL_QUERY,
    result_formatter=format_record,
```

```
)
print('VectorCypherRetriever initialized!')
```

Now we have what we need for our complete pipeline. Again we capture the user question. Create embeddings for the question. Retrieve information from Neo4j using the vector index along with the additional context providing Cypher query. And finally pass these results along with the original question to the LLM for processing so that a complete response can be returned to us.

```
rag = GraphRAG(llm=llm, retriever=vector_cypher_retriever)

query = "What are the primary illnesses mentioned in the doctor's notes?"
response = rag.search(query, retriever_config={'top_k': 5}, return_context=True)

print(f'Query: "{query}"\n')
print('Answer:')
print(response.answer)

print('\n\n=== Enriched Context ===')
for i, item in enumerate(response.retriever_result.items, 1):
    meta = item.metadata or {}
    print(f'\n[{i}] Score: {meta.get("score", 0):.4f} | ClaimId: {meta.get("claimId", "N/A")}')
    print(f'    ClaimStatus: {meta.get("claimStatus", "N/A")} | PatientName: {meta.get("patientName", "N/A")}')
    content_preview = str(item.content)[:200]
    print(f'    Text: {content_preview}...')
```

Output

Query: "What are the primary illnesses mentioned in the doctor's notes?"

Answer:

Based on the provided clinical notes, the primary illnesses (diagnoses) mentioned are:

1. **Chest pain, unspecified** (ICD Code: R07.9)
2. **Gastro-esophageal reflux disease without esophagitis** (ICD Code: K21.9)
3. **Type 2 diabetes mellitus without complications** (ICD Code: E11.9)
4. **Low back pain** (ICD Code: M54.5)

=== Enriched Context === ...

Aura Agents

[Neo4j Aura Agent](#) is a no/low-code agent platform that allows you to build, test, and deploy GraphRAG agents contextualized by your own knowledge graph in AuraDB.

Aura provides the orchestration layer as well as the integration with GenAI models/LLMs and offers different iterative tools that allow your agent to provide the right context to the LLM.

You can test the agent and iterate on it directly in the UI.

Once you are satisfied with your agent, you can either share it internally with other members of your Aura project, or you can make it available externally via either an Aura API endpoint or an MCP server endpoint and thus integrate it into your GenAI application or model.

Navigate to Agents in the left hand menu pane.

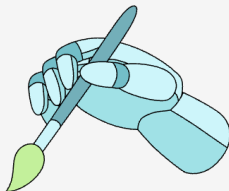
Data services

 Instances

 Graph Analytics

 GraphQL Data APIs

 Agents New



Create your first Agent

An agent is capable of answering complex questions leveraging LLM advanced reasoning capabilities grounded by data in knowledge graphs.

You can also [import an Agent](#) directly.

Create from scratch

 Create with AI

Let's create our first Agent. Click 'Create with AI'

Create with AI



Neo4j AI will analyze your prompt and the schema of the selected instance and generate a custom Aura agent with a list of recommended tools. After the agent is created, you can adjust it to suit your needs.

Instance

[aven]-workshop-dev

Change

☒ This instance contains vector embeddings

Embedding provider

Vertex AI



Embedding model

gemini-embedding-001



Vector Dimensions

3072

Min

Max

Prompt

Create a Health Claims Knowledge Assistant responsible for Claims analysis, provider search, and patient similarity at HealthyHeart Systems.

Skills Analysis example questions:

-How many Claims do I have for diabetes?

-Can you summarize Providers by the types of Diagnosis they have made and Procedure they've recommended?

For best results, include details such as: your use case/context, the goal you want this agent to achieve, the tasks it should perform, any constraints or preferences.

Enter the values on the form.

For the prompt, enter the following:

Create a Health Claims Knowledge Assistant responsible for Claims analysis, provider search, and patient similarity at HealthyHeart Systems.

Skills Analysis example questions:

-How many Claims do I have for diabetes?
-Can you summarize Providers by the types of Diagnosis they have made and Procedure they've recommended?

Patient Similarities: ex: Who is most similar to Thomas Martin?

Doctors & notes: ex: Which Providers have provided note Chunks that support the Diagnosis and Procedures?

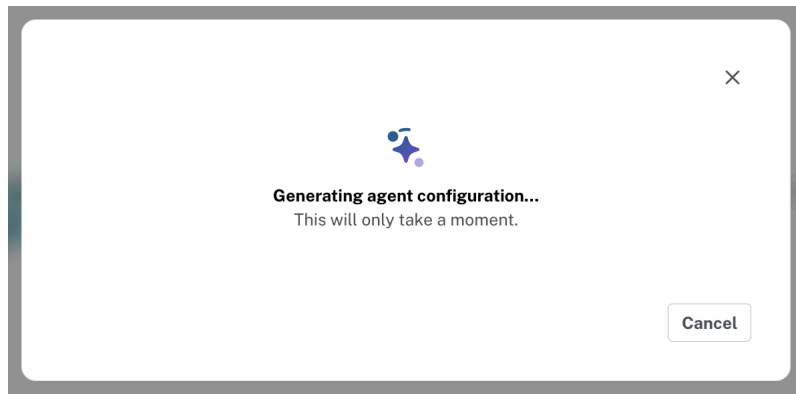
You do not need to ask user permission before using a specific tool. It is implied that you have permission. The user cannot offer expert guidance into what tools to use.

It is important to have Cypher templates to judge similarity through skills and accomplishments/projects in the graph rather than using similarity search. Also ensure aggregation and counting questions do not leverage similarity search but the other tool options instead.

Text2Cypher should be a very last fallback option - only for dead simple aggregation queries. To ensure we have enough coverage in cypher templates, let's also create cypher template aggregation queries for counting Providers on specialty and providerType.

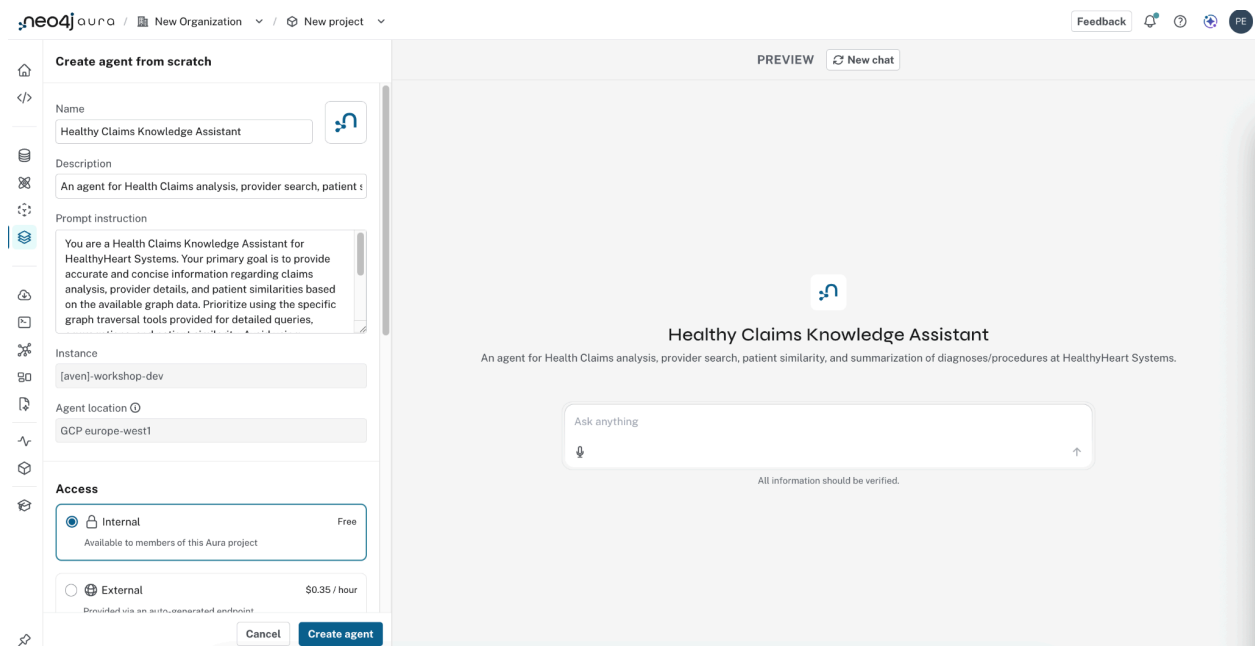
[Cancel](#) [Create](#)

Click the Create button



The agent will be generated...

You will then see the agent created with a surface area where you can use as well as configure it.



The agent has not been saved yet. **Click 'Create Agent' to save!**
Once saved, you can select it or select Configure from the options menu.

Agents

+ Create agent

↓ Import agent

Shared in project ^



Health Claims Knowledg...

[aven]-workshop-dev

Free



New Chat

Configure

Export as JSON

Copy as JSON

Delete

Scroll Down to See Agent Tools.

After any changes now, be sure to click 'Upgrade Agent' to save your changes.

neo4j aura / New Organization / New project

Feedback

Create agent from scratch

PREVIEW New chat

Agent tools

The agent will use its tools to perform actions on the instance

Add tool

- ☒ Count Claims for Diagnosis
Cypher Template
- ☒ Summarize Provider Activity
Cypher Template
- ☒ Count Providers by Specialty
Cypher Template
- ☒ Count Providers by Type
Cypher Template
- ☒ Find Providers with Notes for Diagnosis
Cypher Template
- ☒ Find Providers with Notes for Procedure
Cypher Template

Cancel Update agent

Health Claims Knowledge Assistant

An assistant for HealthyHeart Systems to analyze claims, search for providers, find similar patients, and identify providers associated with notes for diagnoses and procedures.

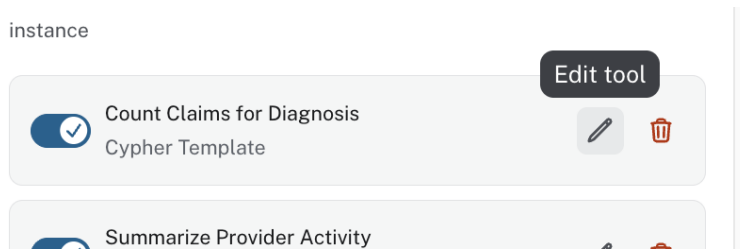
Ask anything

All information should be verified.

Aura Agent has three tools available:

- Cypher Template
 - Executes pre-defined, parameterized Cypher queries against the database and returns the results to the agent
- Similarity Search
 - Uses vector similarity to do a semantic search
- Text2Cypher
 - Transforms natural language questions into Cypher queries
 - Best for:
 - Questions that can't be predicted in advance
 - Structural queries like aggregations

Click on the pencil of one of the Cypher Template tools to edit it.



The edit template view allows us to make any updates to the Cypher. Once satisfied with our query, we can navigate to the Test pane.

Edit Cypher Template tool

ConfigureTestNew

Name

Count Claims for Diagnosis

Description ⓘ

Counts the number of claims associated with a specific diagnosis description.

Parameters and Cypher query

> Generate draft with AI

Draft a Cypher template and parameters from plain language.

Parameters (optional)

+ Add parameter

Parameters

SdiagnosisDescription

The exact description of the diagnosis to count claims for.

Cypher query

Type \$ to access your parameters

1 MATCH (d:Diagnosis)-[:INCLUDES_DIAGNOSIS]-(c:Claim) WHERE d.description CONTAINS \$diagnosisDescription RETURN count(c) AS claimCount

No test run yet

Configure the tool, then run a test to preview results.

Go to Test

Cancel

Update tool

On the test pane, we can run a test to validate results.

Edit Cypher Template tool

ConfigureTestNew

Tool input

Reset values

Name

Type

Value

SdiagnosisDescription

string

infection

Query

Prefilled from Configure. Edits apply only to this test run.

1 MATCH (d:Diagnosis)-[:INCLUDES_DIAGNOSIS]-(c:Claim) WHERE d.description CONTAINS \$diagnosisDescription RETURN count(c) AS claimCount

Test query

GraphTableRaw

claimCount

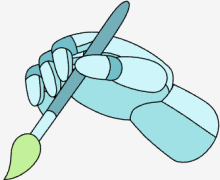
13

Click Upgrade Model to exit from the configure screen and save changes.

Now in a room full of people each creating a similar agent and using the same prompt, I can assure you the results will vary as LLMs are non-deterministic. We are using the LLM to aid us in creating the agent. The LLMs still require guidance and human review.

We can export our agents as json and import them into other Neo4j Aura instances. Delete or rename the agent we’ve just created. You can rename the Agent in the Configure view.

If you delete, the agent screen takes us back to our original options. Click the link to 'import an Agent'



Create your first Agent

An agent is capable of answering complex questions leveraging LLM advanced reasoning capabilities grounded by data in knowledge graphs.
You can also [import an Agent](#) directly.

Create from scratchCreate with AI

If you rename the Agent, you'll find the Import Agent button available above the agent you've just renamed. Click the 'Import agent' button'

+ Create agent↓ Import agent

Shared in project ^



My First Knowledge Assi...

[aven]-workshop-dev Free

🔒 ...

Both options take you to the Agent import form.

Import agent

×

Instance

[aven]-workshop-dev (connected) ▾

Browse file

Clear

Upload, drag & drop or paste an agent config

Navigate to the `agents/Health Claims Knowledge Assistant.json` file in the project and select it.

Import agent

Instance

[aven]-workshop-dev (connected)

Browse file

Clear

```
{
  "name": "Health Claims Knowledge Assistant",
  "description": "An assistant for HealthyHeart Systems to analyze claims,
search for providers, find similar patients, and identify providers
associated with notes for diagnoses and procedures.",
  "system_prompt": "You are a Health Claims Knowledge Assistant for
HealthyHeart Systems. Provide comprehensive and accurate information for
claims analysis, provider search, and patient similarity. Present findings
clearly and directly.",
  "is_private": true,
  "is_mcp_enabled": false,
  "tools": [
    {
      "name": "Count Claims for Diagnosis",
      "description": "Counts the number of claims associated with a specific
diagnosis description.",
      "enabled": true,
      "type": "cypherTemplate",
      "config": {
        "template": "MATCH (d:Diagnosis)<--[:INCLUDES_DIAGNOSIS]-(c:Claim)
WHERE d.description CONTAINS $diagnosisDescription RETURN count(c) AS
```

Cancel

Import agent

Click 'Import Agent'

If you renamed your original Agent, you will now see two available Agent options.

- 

Health Claims Knowledg...

[aven]-workshop-dev Free

 ...
- 

My First Knowledge Assi...

[aven]-workshop-dev Free

 ...

How many claims are there for diabetes?



All information should be verified.

Select the Health Claims Knowledge Assistant Agent we've just imported. And ask a question...

Thought for 3 seconds

There are 14 claims for diabetes.



The response and reasoning are provided. Expand the thought response pane by clicking the down arrow on the right of the Thought timing to expand and validate the reasoning.



Thought for 3 seconds

 Reasoning

My thinking process goes like this: I recognize that the user wants to know how many claims are associated with "diabetes." My internal knowledge base immediately points to the 'Count_Claims_for_Diagnosis' tool as the most relevant function for this task. I then check the requirements of this tool and see that it needs a precise 'diagnosisDescription'. Given the user's explicit mention of "diabetes," it's straightforward for me to pass "diabetes" directly as the 'diagnosisDescription' argument to the 'Count_Claims_for_Diagnosis' tool. This ensures I'm fulfilling the request accurately and efficiently.

 Applying agent tool: Count Claims for Diagnosis

</> Input

{ "diagnosisDescription": "diabetes" }

</> Output

{ "keys": ["claimCount"], "records": [{ "claimCount": 14 }], "summary": {} }

Tokens usage ⓘ

Invocation ID 

There are 14 claims for diabetes.

DIA will inform you which tools were used to retrieve data and generate the response.

Making Agents Public

There are two different ways to interact with your agent externally. You can integrate your agent into your GenAI application by making it publicly available via [REST API](#) from an endpoint, or you can make it available to an MCP client via an [MCP server endpoint](#). The options to do this are in the Configure pane of the Agent.

Access

☐  Internal Free
Available to members of this Aura project

☒  External \$0.35 / hour
Provided via an auto-generated endpoint

☒ Enable MCP server

Note: Both of these methods will start incurring charges.

CONCLUSION

Congratulations! You've accomplished a lot! Completing this workshop included

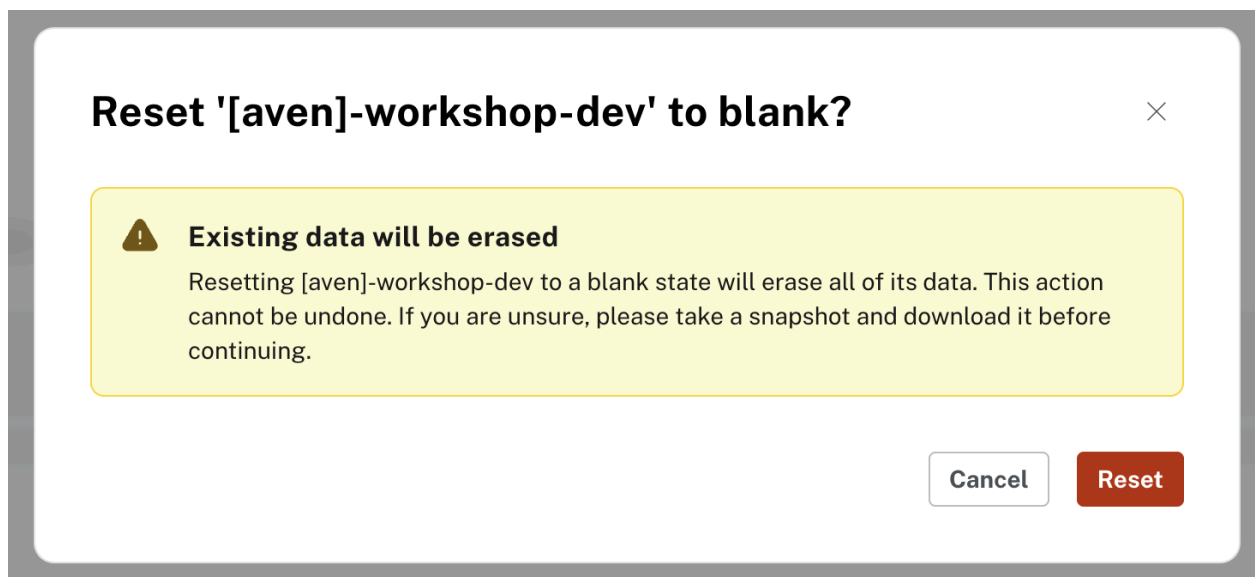
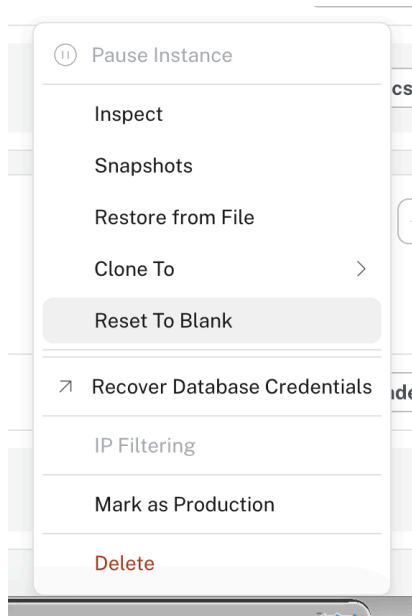
- Importing unstructured content into a knowledge graph in Neo4j using Document Intelligence.
- Importing structured data, connecting it to the unstructured data in the graph using LOAD CSV
- Querying the connected data using Cypher and Python and Vector Indexing
- Creating a complete pipeline for GraphRAG processing using VectorRetriever and VectorCypherRetriever using the Neo4j GraphRAG for Python library
- Creating a Health Knowledge Assistant Agent using Aura Agent
- Learning about Bloom! And other applications available in Neo4j Aura

APPENDIX

RESET Database

In case we have to reload, we have to delete all data and remove constraints before reimporting using Document Intelligence. This is only a requirement of Document Intelligence in preview mode. This limitation is not planned for GA.

Navigate to Instances, and reset the database.



Vector Embedding ('chest pain'):

```
[-0.010486765, -0.023660203, 0.011853571, -0.058251645, 0.008701041,
0.017029656, 0.010226585, 0.029214952, -0.006079374, 0.02581844,
0.0040694918, -0.02039298, 0.0006588761, 0.003017913, 0.10136851,
-0.00968788, -0.004680762, -0.002651188, -0.010978651, -0.02352473,
0.011033404, -0.0068151755, -0.0016743751, -0.008942003, 0.017510308,
0.0063150125, 0.014536074, 0.0018040005, 0.03646075, 0.020492936,
0.014807287, -0.014802581, -0.01592173, -0.0007753263, 0.0050243614,
0.014125602, 0.0030902245, -0.029880099, 0.010551216, 0.0061329883,
0.01857493, 0.010463043, 0.007705833, 0.013355509, -0.013163796,
-0.03404783, -0.009318339, -0.0033506548, 0.012018771, 0.017907862,
-0.027016705, 0.010995463, -0.016040972, -0.16318627, -0.0033845927,
-0.011007312, 0.0008175528, -0.010693294, 0.012686078, 0.0028920264,
-0.016702862, 0.0033221545, -0.032403555, -0.020489726, 0.003764239,
0.006466296, 0.004232694, 0.010129, -0.019478783, 0.0012129915,
-0.010402498, 0.03869572, 0.009106332, 0.0035273898, 0.00779403,
0.02027671, 0.0048557674, 0.018498903, -0.009118166, 0.027653985,
0.006458729, -0.009721032, -0.030551659, -0.020599585, 0.007640532,
0.005824678, -0.016472133, 0.0008690371, -0.0016831815, -0.006491872,
-0.019259905, 0.024615608, 0.030367427, 0.0019416417, 0.005636132,
0.00743821, 0.019172145, 0.019864948, 0.014529151, -0.010549446,
-0.0133599145, 0.013403837, -0.019716134, -0.0042678188,
0.0041598775, -0.02349971, 0.01429449, 0.02033704, -0.00027244526,
0.011670172, -0.0007948063, 0.019003501, -0.026875585, -0.008007566,
0.009350653, -0.18476184, 0.008018877, -0.025261749, 0.009047852,
0.010357534, -0.0026572766, -0.009375524, 0.00041083895,
-0.004950311, -0.021049635, -0.023169689, 0.02115931, -0.001262631,
0.0125460215, 0.005220779, -0.019907195, -0.01602365, 0.018250613,
-0.00018499595, -0.0157233, 0.019032856, -0.006112214, 0.011210627,
0.011851839, 0.005420811, -0.02019902, -0.03429187, 0.008538856,
-0.01171057, -0.01061172, -0.007412292, 0.003856613, 0.01141902,
0.006038867, 0.022057839, -0.029680703, -0.0025818076, 0.00040062083,
-0.00234349, 0.027988795, -0.039999384, -0.0114595685, -0.0029708098,
0.0019571856, -0.032041986, 0.03522228, -0.024460562, 0.0001227631,
0.017311875, -5.9674425e-05, -0.00043468294, -0.011487025,
0.005462235, 0.03129303, -0.007120119, -0.026634453, 0.0027628536,
-0.026530115, 0.003542268, -0.0053119576, 0.0077756373, 0.01974895,
```

0.015986854, -0.005723474, -0.010641775, 0.007887063, -0.030513661,
-0.012898475, 0.0065883053, -0.0056447163, 0.016877355, -0.01939552,
0.021028766, 0.0042066136, 0.01824403, -0.005897079, 0.0043466683,
0.012069612, -0.02979363, -0.008154701, 0.0073227836, -0.015379711,
0.016865375, 0.02512048, -0.022297379, 0.007958414, 0.0065704803,
0.00031373466, 0.03153307, -0.0035461695, 0.007640151, 0.013036483,
0.0048157945, 0.01859545, 0.01133341, 0.036118656, 0.003601076,
0.023843616, -0.016517503, 0.0020403245, -0.02832574, -0.0063831313,
0.01109771, -0.00012004299, 0.0038090358, -0.0088364165,
-0.0056968983, 0.020766025, -0.00803083, -0.0026846475, 0.005887174,
0.029927796, -0.0008157152, -0.0018821994, 0.0066376845,
0.0033034217, -0.002689906, 0.011375589, 0.008483153, -0.011068272,
0.009670586, -0.008465081, -0.0018069202, 0.023495134, 0.004304629,
0.010213385, -0.0069195083, -0.034403503, -0.007054821, 0.0058679045,
0.032517377, 0.0008446435, -0.008340292, -0.0052852854, 0.008064118,
-0.0011202253, -0.0029154662, -0.006504468, 0.0014594785,
0.009217425, -0.005755231, -0.004181274, -0.015672695, -0.0139817335,
0.0075349235, -0.012228299, -0.02301875, -0.0075997827,
0.00038772705, -0.024109097, 0.01341457, 0.015244652, 0.00596888,
0.0095318025, -0.0144851785, -0.009002243, -0.027325124,
-0.0013207062, -0.008671605, -0.037031237, 0.032819387, -0.10772687,
0.0010431872, -0.0039729737, -0.014290845, 0.0016355198, 0.002805205,
0.006200302, -0.017530687, -0.029521845, 0.01026935, -0.032695193,
-0.0081421705, 0.036604833, -0.0077050165, 0.0053942413,
0.0044104015, -0.029706014, -0.00085833325, -0.0146664595,
0.0055181324, 0.006082474, 0.016411047, 0.0021845554, 0.019248586,
-0.0048365495, -0.016795127, -0.00093228294, -0.01619605,
-0.007394696, -0.007593387, 0.019471457, 0.0139342295, 0.009889249,
0.00037995275, -0.018752767, 0.0045962506, 0.013891854, 0.006078297,
0.016824156, -0.04175839, -0.0036586411, 0.0036384703, -0.015728597,
-0.009805078, 0.0039585778, -0.0056995833, -0.011003449,
-0.0136794485, -0.017401863, -0.01212389, 0.00782266, -0.009930178,
-0.004773703, 0.021464113, 0.0039013226, -0.016998861, 0.029110355,
0.0070998026, -0.027236935, -0.0036852425, 0.013627503, -0.013937734,
-0.0150232585, 0.039698735, 0.012434866, -0.01953723, -0.0264056,
-0.011213808, 0.007067861, -0.03393652, 0.004980905, -0.0071133235,
-0.014839913, -0.032235526, 0.03730395, 0.0032329177, 0.021504913,
0.00053380267, -0.010437395, 0.012595404, -0.00870057, -0.009009368,
0.003400716, 0.034448113, 0.020707823, 0.012391831, 0.01053209,
0.007064259, 0.012953913, -0.020042868, 0.014720553, -0.010904182,
0.00074208254, -0.02383054, 0.008208448, -0.00053940865,
-0.037571162, 0.0045842095, -0.0014500715, 0.018809974, 0.0065363105,

0.005732242, 0.023225902, -0.021254221, -0.0017301582, 0.032752495,
-0.008781503, -0.0013312264, -0.010704679, -0.0025054135,
0.015291914, -0.006625965, -0.00016709372, -0.0035154168,
-0.016629895, -0.017164975, 0.0052320687, -0.00827465, 0.015733967,
-0.008518416, -0.031085478, 0.018129082, -0.009458152, 0.006220072,
0.0033238987, 0.009722298, -0.010914279, -0.013419817, -0.038622,
0.0083302455, -0.022213807, -0.023404172, 0.024157008, 0.013844961,
0.021892006, 0.04291468, 0.017747227, -0.0021303261, -0.0053018993,
0.0021058507, 0.001321641, -0.0041862708, 0.031674724, -0.0052881557,
0.019100603, -0.006228117, 0.004136431, -0.025108693, -0.013639854,
-0.023002066, -0.019750966, -0.009972107, -0.0022032785, 0.018777719,
-0.009024669, -0.0034402672, 0.0002744884, 0.004430109, -0.02065709,
-0.017986197, 0.010374654, 0.017206054, -0.014344053, -0.028494805,
0.028906733, 0.014752336, 0.0093651, 0.019727869, -0.031024845,
0.012472339, 0.003924694, 0.0061009834, 0.008199678, 0.0053031542,
-0.0004958631, -0.013210986, 0.020371107, 0.0036843433,
-0.00087511685, -0.023885632, -0.003323628, 0.01821306,
-0.0045440746, -0.029552117, -0.011574967, -0.033834375,
-0.0066901827, -0.005184238, -0.016463293, -0.0045520635,
-0.008961951, -0.019765927, 0.00706533, 0.0026264398, -0.0004988826,
0.017197488, 0.00056352303, 0.0154794, -0.0044432795, -0.015224157,
0.0038767692, 0.0362287, 0.01836319, -0.006669186, 0.003998641,
-0.0022859555, 0.0021090787, 0.024499122, 0.018053008, 0.01600084,
0.019843182, 0.004041453, 0.00014287874, -0.022220062, 0.021030493,
0.012219235, -0.014782603, 0.0033621034, 0.020730473, -0.032976665,
0.0027114055, 0.022318974, 0.018542215, 0.0020465618, 0.009584643,
-0.007834972, 0.0047963616, -0.0030012277, 0.012869985,
-0.0066394205, 0.018351557, 0.0013436383, 0.024801811, 0.016776733,
0.0011947824, 0.008605335, -0.008899964, -0.0138641605, 0.0003106246,
-0.01404207, 0.0015202926, 0.029391512, 0.011988793, 0.0013770312,
0.013484171, 0.0035667913, 0.007803694, 0.02955475, 0.011632207,
-0.010417975, 0.031046128, 0.010234388, 0.025612786, -0.010050424,
-0.002009042, 0.012077797, 0.0027790251, -0.005677872, 0.009969141,
-0.013292984, -0.010324114, 0.0058447635, 0.012221339, -0.022214888,
-0.0054860483, 0.0025240732, -0.0020705021, 0.02466549, 0.015441632,
-0.1244179, 0.03769271, 0.0052675945, -0.0027267095, 0.021914488,
-0.021797366, 0.0019188737, 0.002735737, -0.0018243184, -0.031849075,
0.00614329, -0.0029849713, 0.003543249, 0.0011279478, 0.0013759686,
0.0023009533, -0.0022887741, 0.015249422, -0.0025809377,
-0.008638544, -0.016779765, -0.014398362, 0.038746398, -0.004462661,
-0.014941162, 0.0041618096, 0.01358332, -0.018177714, 0.008793777,
0.043102417, -0.028344952, -0.025297107, 0.004699551, 0.0044401577,

0.011583101, 0.0029720482, -0.004690942, -0.01356265, 0.026155606,
0.011862087, 0.0026463892, -0.004397433, -0.0077439896, 0.019338805,
-0.016817512, -0.006144765, 0.010538593, -0.0069993874,
-0.0023884056, 0.00040848262, -0.010503778, 0.0214433, -0.004461817,
0.006642346, -0.009306463, -0.0022611988, -0.0016304225,
0.0009556385, -0.0116410125, -0.009600787, -0.02649963,
-0.0041035623, -0.008120471, 0.015328557, -0.008467863,
-0.0036538923, -0.009192908, -0.0110872425, 0.024178496,
-0.0032894544, -0.0146486135, -0.0020239183, 0.03163783, 0.005755357,
-0.0014419605, -0.01923308, -0.0037217399, 0.030943666, 0.021087954,
0.032114543, 0.009880331, -0.016286243, -0.054253723, 0.017613424,
-0.01659718, -0.01910244, 0.002682447, 0.0058986554, 0.008468065,
0.0065253186, -0.0006690911, 0.00588787, 0.022363544, 0.03013274,
0.029069372, -0.012733215, 0.0046710847, 0.000106781095, 0.023038097,
0.013708091, -0.026282359, 0.011473537, -0.017105686, 0.0043359823,
-0.006057743, -0.006259072, -0.0030086762, 0.007653313, -0.023959646,
0.031938955, -0.0060881013, -0.0050171437, 0.0132110035, -0.127291,
-0.011856209, -0.0043328563, 0.021089857, 0.021219552, 0.00474895,
-0.009277328, 0.007331595, -0.019031309, 0.037099484, 0.0026948773,
-0.017975364, 0.0040284344, 0.00017987142, 0.035600822, 0.16910453,
0.012302262, 0.010936352, -0.00739325, -0.021284012, -0.009219178,
-0.012263163, -0.006695048, 0.0058185053, 0.003291703, 0.018135294,
0.010490948, -0.009528653, -0.039864045, -0.0042577633, -0.012918944,
-0.002770357, 0.012563026, -0.0057418095, -0.0324085, -0.0031721587,
-0.008058982, -4.6678142e-05, 0.023332058, -0.009648058, 0.027184268,
-0.010256355, 0.020795094, 0.006519337, -0.013066589, -0.025258875,
0.01592205, 0.0046448964, -0.01577785, -0.019267103, 0.010815596,
-0.032913297, -0.013463886, 0.015919348, -0.022008028, 0.00084207865,
0.011111325, -0.004584742, -0.032507733, -0.0034833117,
-0.0010038174, 0.044633005, 0.024749545, 0.00410604, 0.009260677,
0.0054520676, 0.029083675, -0.00036091887, 0.009742427, 0.025526835,
-0.019850558, 0.000567444, -0.00487319, -0.011439575, 0.001550812,
0.015628206, -0.009073002, -0.029384263, 0.03316732, -0.02180179,
0.024949888, 0.0069111832, 0.017567309, 0.0028994286, -0.009594209,
-0.0018667702, -0.028124342, 0.025521107, 0.014779813, -0.019279806,
0.017340088, 0.00020786602, 0.01299097, 0.0018005486, 0.010436851,
-0.0027160605, -0.015574465, 0.043513097, 0.004603067, -0.0003266774,
0.0016791811, 0.0028778943, -0.016037133, 0.016457431, -0.012762759,
-0.0178613, -0.029702794, 0.0061572623, 0.005590419, -0.0068761897,
0.02017516, 0.012628647, -0.00054958084, -0.011798481, 0.0065376046,
0.0075935475, 0.01664684, 0.0023110018, -0.0008556904, -0.0069518993,
-0.00054195884, -0.0014070289, -0.014661917, 0.01537554, 0.013548709,

-0.005090081, -0.009514417, 0.006938943, 0.0063300417, -0.0193177,
-0.021197047, 0.0029835529, 0.014491713, 0.0030109133, -0.0119796535,
-0.006693802, 0.006548828, 0.0015549592, 0.0045573586, 0.016175475,
0.004687511, -0.0021596262, -0.0053828955, -0.0012248337,
0.011942997, 0.012682245, -0.039747853, 0.0070584877, 0.008906489,
0.0009487865, 0.015605279, 0.0017314506, 0.008480651, 0.0038950038,
-0.013953854, -0.01706988, 0.0028355925, 0.005516824, -0.01700731,
0.004685564, 0.009133191, 0.0068359203, -0.037105385, 0.010314883,
0.009905575, 0.007009183, 0.0056439666, -0.022528078, 0.0011337261,
-0.025039751, 0.0066345576, 0.008039071, -0.0074249995, -0.02487887,
-0.012510176, -0.0031747445, -0.009351487, 0.008612241, 0.007914267,
0.0073202723, 0.003532179, 0.0135726975, 0.0161359, 0.0014563971,
0.0014529377, 0.008214329, -0.009826826, -0.012568497,
0.000109341396, -0.017403435, -0.009143276, -0.00551262, 0.003781657,
-0.011253449, 0.008843367, 0.005316385, -0.0015695001, -0.018613523,
0.016227337, 2.5595818e-05, -8.445281e-05, 0.012864567, -0.002471486,
-0.012481325, -0.016303487, -0.003784893, -0.0039109695,
-0.0034931775, 0.00061938376, -0.0054186704, 0.007152363,
0.006587307, -0.003206615, -0.0012942707, -0.005649997, -0.011786234,
0.0079793725, 0.001063971, -0.0011211064, 0.003828808, 0.006750728,
0.010331271, 0.0041370117, -0.0032978507, -0.01420781, 9.155161e-05,
-0.012418489, 0.009163736, 0.008695162, -0.0042543453, 0.0022614659,
-0.017210891, 0.017070793, 0.010829532, 0.013517084, 0.003790252,
0.0012797173, -0.0012684667, 0.0050237537, -0.004289162, 0.018287186,
-0.008738263, 0.009450978, -0.014789199, -0.01092114, -0.0031947915,
0.00912567, 0.011706722, 0.004503323, 0.010594884, -0.013345899,
0.00038828558, 0.009860358, 0.0030688618, 0.009136039, -0.0067585614,
-0.0024327585, 0.01683629, 0.008579069, 0.00013893608, -0.0034544095,
0.0020714004, 0.0114989765, -0.0023134563, -0.0069672884,
-0.0010260855, 0.002156196, 0.00028259296, 0.007994807, 0.022231536,
0.0018256258, 0.00685014, -0.0026571609, 0.007320908, 0.02184859,
-0.0041907583, -0.009537058, -0.025695046, -0.007907826,
0.0017082898, 0.0020626835, 0.010608578, -0.0018963345, 0.0047762413,
-0.0015347786, 0.011494702, -0.010025848, 0.0030021158, 7.144592e-05,
-0.0012039072, -0.017894551, 0.0027584268, 0.0013729738,
-0.0029634426, -0.0024643056, -0.015428296, -0.012326489,
0.0036225738, -0.00402748, -0.012729352, -0.0070986543, 0.004476274,
0.00040140573, -0.0024754852, 0.010283854, 0.007325372, 0.0029404976,
-0.0062667835, 0.00804799, 0.0034863048, 0.09198502, 0.0075793033,
0.012571791, 0.010687174, -0.010469868, -0.0010492196, -0.0011048621,
-0.0058716647, -0.00809244, -0.004415691, -0.0030431012, 0.009776293,
-0.010666377, 0.01656981, 0.011102495, 0.007268022, -0.011202483,

0.009915125, 0.0126847215, -0.004839519, -0.006304172, 0.013126134,
0.001331039, -0.012229749, 0.008748641, -0.021305153, 0.0056001395,
-0.021482676, 0.0023053868, 0.010747846, 0.0003005119, 0.005980454,
0.0052702697, -0.0010436308, -0.016852949, 0.0019714206,
-0.0042918697, -0.0107487505, 0.006044181, 0.008840517, -0.011993704,
0.013497299, 0.008151892, 0.012903625, 0.003338361, 0.006738416,
-0.010833563, 0.007849663, -0.007035116, -0.015370021, -0.020642167,
0.0016183372, -0.016851041, -0.008449301, -0.0051014656,
0.0027374171, 0.0069377413, 0.006344336, 0.0023228712, -0.0027080819,
0.010344473, -0.009539979, -0.021807369, -0.006417634, -0.012794953,
-0.014484714, 0.000957616, 0.0033076047, 0.001460641, -0.0069792154,
-0.006338724, -0.0054063285, -0.006803792, -0.004033752, 0.04978116,
0.016429394, -0.008579508, -0.0053630616, 0.0010792539, -0.010612994,
0.020968163, 0.019915594, -0.022512086, -0.0065805037, 0.0023844598,
0.010083933, 0.014654611, 0.00338922, -0.0041280016, 0.0035897505,
0.01882377, -0.011894919, 0.0056772404, 0.010151732, 0.010274903,
0.011612734, 0.10363516, 0.019681463, -0.018977087, -0.011424413,
-0.0014622307, -0.021904595, 0.0076058307, -0.0030731417,
-0.006162798, 0.0062493943, -0.002574787, 0.00093430077,
-0.017804889, -0.0042228927, -0.015701206, 0.0017831685,
-0.006019259, 0.0061232788, 0.0016456797, -0.0001804407,
-0.005450897, -0.0032481658, -0.0068921717, 0.02611849,
-0.0015322272, 0.004278726, 0.007393488, -0.010134222, -0.0029725644,
0.00038838282, 0.010940955, -0.011524146, 0.0053861435, -0.013557122,
0.013530785, 0.01632266, 0.013544431, -0.0002921138, -0.005256572,
0.006910748, -0.011740668, 0.0034279753, -0.015656238, 0.0099824825,
-0.009820518, -0.0044072834, -0.016480302, -0.02138339, 0.0015209238,
0.011563062, 0.017271006, 0.02199949, 0.00039497376, 0.009681717,
-0.00955069, 0.015366126, -0.024321554, -0.0053792703, -0.015498733,
-0.0094038965, 0.012713862, -0.0022999144, 0.002012393, 0.008576197,
0.00884054, 0.01680084, 0.0068226825, -0.010957029, -0.0018606995,
0.015846504, -0.0035279682, -0.00052200374, 0.02526271, 0.010676671,
0.008409808, 0.0021510073, 0.015567265, -0.0074415593, -0.0036099274,
0.0010150117, -0.0072277826, 0.0017428236, 0.0012700642, 0.011445852,
0.012197182, -0.0008883852, -0.021389984, 0.011272283, 0.0046244403,
-0.019443884, 0.020086924, 0.0044728527, -0.009809362, -0.0025452964,
0.008613666, -0.016609611, 0.004986981, -0.006253027, 0.0052691833,
0.02450223, 0.013649408, -0.0073841056, -0.017441923, 0.018333167,
-0.010400855, 0.009080183, -0.005299041, 0.0041053197, 0.003393759,
0.01134812, -0.0003419475, 0.0074433256, 0.0031061147,
-0.00085969374, -0.0022763843, -0.005839449, 0.00754745, 0.019696247,
0.0007077084, -0.016601328, 0.0109627955, 0.012992457, 0.004826927,

0.005878252, 0.00054069294, -0.0118598575, -0.006288068,
-0.0004893515, 0.0008162752, 0.007528609, -0.007830347, 0.0030421896,
0.001262405, -0.0012521086, -0.0077159675, 0.0078246435,
-0.0032446044, 0.0046581523, -0.013186289, 0.012173406, -0.009949581,
-0.0049063466, -0.006083897, 0.0004524515, -0.01606718,
0.00046453686, 0.00017453078, -0.023108369, -0.006733099,
-0.0022017248, 0.011974946, 0.0064081135, -0.010937821, 0.008338477,
0.039291013, 0.017467564, 0.01301659, -0.0013972064, 0.006014428,
-0.0026391759, 0.014371591, -0.051330436, 0.003395675, 0.0034550396,
0.008659534, -0.019948168, -0.0051968694, -0.0115546435,
-0.009596289, 0.01641766, -0.010148558, -0.019330991, 0.0039484305,
-0.013873383, -0.00293883, 0.0014213187, -0.006999606,
-0.00018524006, -0.0011776565, -0.011492751, 0.0061820145,
0.0027499418, 0.0011785869, 0.009314104, 0.002175849, 0.0138329705,
0.015355627, -0.013982242, 0.010911884, -0.00085561763, -0.023834681,
-0.010776304, -0.016809613, -0.001787308, 0.012112858, 0.0023548547,
0.003758022, 0.025887694, 0.0028158245, 0.00047683314, 0.01480565,
0.010404627, 0.0038957712, -0.013857213, -0.0026576272,
-0.0065186266, -0.018750707, 0.010223619, -0.0029987618,
-0.012023248, -0.012200075, 0.015312515, 0.0075503467, -0.005177308,
0.022145426, 0.014314994, -0.014523706, 0.002835386, 0.0032964698,
0.009762083, 0.0039515523, 0.023469523, -0.007486963, 0.008933473,
0.004876127, -0.020180844, 0.0059684855, 0.020327214, 0.0070654145,
-0.0051769298, 0.011513798, 0.009876302, 0.01041885, 0.0020013398,
0.002586465, 0.004004315, 0.003947428, 0.019426648, 0.0063621188,
-0.013886401, 0.0014804238, 0.0027263567, 0.017687414, -0.0015238377,
-0.012145634, -0.008398317, -0.020783905, 0.0012211258,
-0.0074769957, -0.008460484, -0.0015593373, 0.006192896, 0.006901963,
-0.0033523543, -0.0014330196, -0.00041873555, 0.012641704,
0.0039583645, 0.0059505175, -0.0006149807, 0.019640258, -0.014990302,
-0.0058729737, -0.020926286, -0.01940956, -0.0014676596,
0.0059650815, 0.019382212, -0.008852434, 0.009171569, 0.005034151,
-0.0014881217, -0.007546407, 0.003744932, 0.006210792, 0.0030592452,
-0.0112334825, -0.005468449, 0.00045120323, -0.0062766247,
-0.025483474, -0.017399663, 0.027273955, 0.00942867, 0.011861997,
-0.006397852, 0.0015145546, -0.017060492, 0.010995502, 0.004929797,
-0.009810123, -0.007342675, 0.011097624, 0.020664873, 0.027478786,
-0.012675352, 0.002808605, 0.012072642, 0.02190574, -0.013983038,
0.00092653924, -0.0052443356, 0.0006984826, 0.0059257685,
-0.010725469, -0.0055321814, -0.0067152153, -0.014151686,
0.0045364024, -0.011921962, -0.0020729313, -0.0016904834,
-0.016516807, -0.003970319, 0.018012831, -0.0069527715, -0.0194503,

0.011692837, 0.0024585694, 0.00090050284, -0.01523028, 0.0032077206,
0.0060626506, -0.0054361084, -0.017741373, -0.009678332,
-0.017089399, -0.012061095, 0.007649354, 0.0013593037, -0.015019403,
-0.0045693666, -0.00054365443, 0.002706905, 0.026263554, 0.008737502,
-0.0009765896, 0.014369789, 0.00063743076, 0.0046503884, 0.008710737,
0.003436545, -0.0063726106, 0.006604072, -0.02174063, 0.011990366,
-0.011487762, -0.01710843, -0.008673491, -0.0111190155, 0.0057715126,
0.019394532, 0.026293674, 0.0025175058, -0.0102687385, -0.0039058528,
-0.0014244752, -0.0034729117, -0.020362632, -0.0022193098,
-0.004932427, 0.0047544097, -0.0069363248, 0.006747811, 0.010077807,
-0.01167285, -0.09304375, -0.008722626, -0.0002642398, 0.003295844,
-0.027287027, 0.019490458, 0.010298483, 0.00083291874, 0.0076178294,
-0.023422433, -0.012299729, -0.010145481, 0.022893775, -0.032350328,
-0.011250282, 0.0017339889, 0.013672863, -0.009088082, 0.0009949801,
0.002507604, -0.004136871, 0.010938612, -0.0040287306, 0.0016399312,
0.0030374401, -0.003896964, -0.002074596, -0.009518502, 0.007179211,
0.0022391947, -0.019576736, 0.004214352, -0.004282386, -0.0001957075,
0.0032792655, -0.012011821, 0.018284857, 0.0035426943, -0.14325766,
0.020924335, 0.017940434, -0.0033449263, -0.0036280528, 0.0027838286,
-0.015054645, -0.002826607, -0.010570083, 0.0059035146, 0.00435346,
-0.011264989, 0.00017536787, -0.00841459, 0.013163351, 0.003839636,
-0.004275903, 0.010535366, -0.005891874, 0.0065519395, -0.0016925408,
0.003232426, 0.0018426849, 0.00048452785, 0.0072152466, -0.016026553,
0.01979978, 0.022005392, -0.0146154035, 0.0041885837, -0.016993469,
-0.026740342, 0.00028028042, 0.000912828, -0.0147155505,
-0.0005463687, -0.0013955128, -0.0002623441, -0.0071327956,
0.013292628, -0.0058903154, 0.0054570907, -0.014878059,
-0.0030627472, 0.011295424, -0.0076652584, 0.0051192893, 0.005967345,
-0.010693501, -0.0059114993, -0.018467257, -0.007389582,
0.0015929866, -0.0005186753, 0.008904702, -0.004828202, 0.006810172,
-0.001580752, 0.0051646037, 0.006408121, 0.00032587312, 0.004266348,
-0.004529505, 0.00017904631, -0.0012987629, -0.012955136,
0.0070453095, -0.013258554, 0.016650625, 0.0070174774, -0.014253545,
-0.012515552, 0.010945355, -0.028572364, -0.0030782635, -0.016740514,
0.011648001, 0.018366424, -0.015433398, -0.005613359, 0.009843434,
0.010236999, -0.0059496854, -0.0049847006, -0.006662322,
-0.013280329, 0.00091193133, 0.0231989, 0.0077153477, -0.028344411,
-0.0043806448, -0.006989827, -0.013000931, 0.0025834881,
-0.0015268766, -0.012906752, 0.007973775, 0.03282353, -1.3074866e-05,
-0.0013522365, 0.009615088, 0.0028823682, 0.014498769, 0.010618198,
-0.018733121, -0.02264126, 0.0014181418, -0.0014772069, 0.017771991,
-0.007763652, 0.009712346, 0.0033424422, 0.017354002, 0.024947997,

-0.0065869284, -0.004282195, 0.016285418, -0.016611597, -0.020333843,
-0.016050061, 0.004330731, 0.012543647, -0.00021978654, 0.004110404,
0.022840606, -0.018896317, -0.011210066, -0.028861152, -0.014536185,
-0.008734973, 0.009324516, -0.025866175, 0.019440802, -0.0059125824,
-0.00015203714, 0.01988531, 0.0039040092, -0.015196657, -0.012763839,
-0.0006356654, -0.01904775, -0.004388727, 0.0004393606, 0.011459464,
-0.021832509, -0.009980658, 0.016469076, -0.0054829647, 0.02646744,
0.011128767, -0.0022598794, -0.0014936744, 0.007920882,
-0.0034978213, -0.01762758, 7.5411146e-05, 0.013865219, -0.009331346,
-0.021417592, -0.006136862, 0.005077098, -0.012176444, -0.009094372,
-0.0023272026, 0.005073711, -0.005654629, 0.0020661633, -0.022699513,
-0.011750761, -0.0012735883, -0.0168354, 0.005931361, -0.005186616,
-0.015755652, -0.0034931889, -0.0059169168, 0.013183559,
-0.0064544096, 0.0029721635, 0.0031917482, 0.00063527876,
-0.0031213625, 0.013880594, -0.0022170823, -0.0027080644, 0.01711086,
0.0037065041, 0.025803735, -0.0025047688, -0.01077972, -0.0140013145,
-0.023831505, -0.014808472, -0.0033631795, 5.7626552e-05,
5.5976558e-05, 0.010818299, -0.024743037, -0.026726602, -0.00325267,
-0.0025908228, -0.0034367107, -0.008566181, -0.12398326,
0.0066477703, -0.006077241, 0.005315565, 0.0066058836, -0.013516498,
0.019681437, 0.009237168, 0.0063250293, -5.326051e-05, 0.0042830976,
-0.010143464, -0.019263223, -0.0062095528, 0.02215518, 0.016601877,
0.0017302865, 0.002983352, -0.0020950057, -0.025765581, -0.009936481,
-0.0008149598, 0.005011025, -0.00049594254, 0.0032123737,
-0.0017316162, 0.0064597256, -0.0031084924, -0.010501061,
-0.00066118996, 0.001107813, 0.00036603143, 0.0134487115,
0.0028995213, -0.02019517, -0.025287954, 0.006395465, 0.0045945332,
-0.01369276, 0.020814683, -0.009735808, -0.016043494, 0.0011388112,
-0.0010159491, -0.0059671635, -0.0024121823, 0.0070848586,
0.010703916, -0.012000489, -0.009532192, -0.0036256006, 0.013573011,
0.004425453, 0.0074666883, -0.007987279, -0.0018002461,
-0.0060933284, 0.022485532, -0.0062783877, 0.016928906,
-0.0004110336, -0.0032591203, -0.0024988358, 0.006051597, 0.01644654,
0.010992128, -0.007430644, 0.15865315, -0.004815093, 0.009905037,
0.00053823274, -0.001850199, -0.0021227924, -0.022907913,
-0.00064380065, -0.007059487, 0.011619744, -0.010622593,
0.0013013716, 0.0012460114, -0.007632963, 0.012191932, -0.017181987,
0.01871038, -0.017739192, 0.025393024, -0.026163658, -0.011376091,
0.009441785, 0.0092008235, -0.009497557, -0.0015834393, 0.0011420636,
-0.011347934, 0.01787357, -0.013248947, 0.0076568746, 0.019541902,
-0.01584381, 0.004359301, -0.012942644, 0.0067358897, 0.012887474,
-0.006083977, 0.02134163, -0.0110032065, -0.0044694724,

0.00045384656, -0.0009079433, -0.018974543, -0.0066562714,
0.008722561, -0.020682167, 0.01638377, -0.0010400811, -0.02587446,
-0.014887944, -0.010666389, -0.004646101, -0.025685651, -0.008506541,
-0.02484599, -0.003093151, -0.0119321095, 0.002443517, -0.008909617,
-2.8056473e-05, -0.0054616136, -0.003973963, -0.015778048,
0.0023424188, -0.0014793217, 0.015067944, 0.019043172, -0.0065936265,
0.012982716, -0.14467634, 0.02196256, -0.0065266257, 0.017228233,
0.01950506, 0.001809355, 0.019243283, 0.019974303, 0.0054654414,
0.012934497, 0.0045305514, 0.019091588, 0.006644133, 0.0005520686,
-0.00025287725, 0.0061843754, -0.0002881513, 0.018590758,
-0.003339625, 0.007183362, -0.0019721638, -0.0071573895,
-0.0016538271, -0.009324231, 0.005904352, 0.006307226, -0.0032826592,
0.0030534985, -0.023760987, -0.011416703, 0.012545193, 0.010185661,
-0.008630676, -0.008391178, -0.021092055, -0.0010195974,
0.0009697195, 0.005129156, -0.026299678, -0.0017971742, -0.021610027,
0.011123948, -0.012431327, 0.011842595, 0.013857523, 0.025983784,
-0.0063345875, 0.0020317307, 0.008867034, -0.013242466, -0.0082422,
-0.0025687192, 0.0020863158, 0.0012388857, -0.0049330257,
0.024544032, 0.01092156, -0.0054228944, -0.016308391, -0.03410887,
-0.00178228, 0.017681355, 0.010799404, 0.0002668938, -0.010089832,
-0.008743196, -0.012427468, 0.018480603, 0.034397624, 0.005519799,
0.0017074534, -0.0036169393, 0.016471611, -0.007430401, -0.017753968,
0.008577443, 1.8560248e-05, -0.0023304871, 0.001730338, 0.011333873,
0.0057377964, -0.0007955487, -0.012102909, -0.004937157, 0.052811276,
-0.0017140832, 0.013856306, -0.0039817803, -0.013199435, 0.008848823,
0.003921916, -0.0058248504, 0.0001622432, 0.0263479, 0.013451378,
-0.009656591, 0.0024909992, 0.024171265, -0.008312315, -0.0065476745,
-0.0108659, -0.018427012, -0.018226381, -0.0019053391, -0.0003560696,
0.02122122, -0.01035498, -0.0036918712, 0.009609423, 0.016198944,
-0.016976103, 0.008688358, 0.01081407, -0.008336273, -0.0050625657,
-0.034170423, -0.012621626, 0.011852522, 0.00508985, 0.021249028,
0.017650103, 0.02539806, 0.0023961558, -0.0063159023, 0.0068543,
-0.0059313555, 0.0010364399, 0.0023236803, 0.00655916, 0.01102746,
0.028834872, 0.028045371, 0.02388966, -0.0039395974, 0.002906736,
0.019055162, -0.0015813583, -0.009989405, 0.0032382475,
-0.0033728518, 0.0031113692, -0.01317979, 0.012272758, -0.013786448,
0.00687682, -0.01407061, 0.00819304, 0.0044915904, -0.015679866,
-0.017740715, 0.01263562, -0.0011499015, -0.007204733, 0.018213028,
0.008035335, -0.029753914, -0.0016721096, -0.012907098, 0.013928912,
-0.0036627052, 0.0035087306, 0.0029170178, 0.008656758,
-0.0067996206, 0.015829727, -0.00041933416, -0.0045657624,
-0.012841477, -0.0071332557, -0.0014549629, -0.008968085,

0.013572264, -0.008245431, 0.01606892, -2.1167001e-05, -0.012524417,
0.0009737435, -0.0018159476, 0.021420516, -0.07565393, 0.0219894,
0.028755123, 0.023021193, -0.0072234995, 0.01546112, 0.016524453,
0.013395677, -0.019554364, -0.014958313, 0.0003844756, 0.005516228,
-0.0046645454, -0.01945644, 0.009452157, 0.004675511, -0.013791559,
0.008199991, -0.0056707757, -0.008403562, 0.0023286662, 0.014869647,
-0.018073201, -0.010317518, 0.00505617, 0.0074446276, 0.01889736,
0.004901418, 0.014187547, 0.014460269, -0.023358764, 0.0008370085,
-0.006956068, -0.01197818, -0.0031287696, -0.013876073, -0.003859961,
0.02460052, -0.013789663, -0.06279469, 0.006399891, 0.025815915,
-0.10422087, -0.00011300223, 0.004253156, 0.007433905, -0.014087253,
0.010838618, 0.008562673, -0.0040894975, 0.009716452, -0.0038656287,
-0.0056184335, -0.0015974339, -0.00630784, 0.019827686,
-0.0055228425, -0.03447097, -0.009397571, -0.016209383, -0.02656007,
-0.00979893, 0.0034209632, 0.0030700841, -0.0062859287, -0.008736561,
0.0023093743, 0.0047439467, 0.022036351, 0.0012181851, -0.010624207,
-0.0010843315, -0.0021108463, 0.014179894, -0.014602993, 0.009561052,
0.010915378, -0.015345177, -0.004267032, 0.0047859033, 0.0066081546,
-0.017385947, -0.005962215, 0.035010148, 0.010329462, -0.0068289177,
-0.011122887, -0.11829046, 0.010260117, -0.0061889784, 0.0018246801,
-0.00043651878, -0.017082818, 0.017336411, 0.10377887, -0.015641749,
0.0032443234, 0.005567029, 0.009459699, -0.022151459, 0.01768739,
0.024507046, 0.020908229, 0.022321597, 0.0049627465, -0.00020111102,
0.009340581, -0.00015794479, 0.0019175999, -0.007106214,
-0.0041533066, -0.0045811986, -0.048889775, -0.004312296,
-0.00620159, -0.00011250951, -0.005371551, -0.008500871,
0.0038943805, 0.0067262445, -0.008814937, -0.004129583, 0.0044988124,
-0.0066934894, 0.005397751, -0.02933859, 0.020067737, -0.0024864953,
-0.000631909, -0.019810712, -0.0024765173, -0.004740718, 0.005420733,
-0.003852421, 0.000951886, -0.0063142865, 0.00015359433, 0.010238375,
-0.016723292, 0.018081127, 0.0006278379, 0.030376567, 0.00074262265,
-0.0100167645, -0.000199236, 0.020471638, 0.0016830515,
-0.0072257016, -0.005812601, 0.0017864292, 0.029095974, 0.0028444726,
0.003347106, -0.02667577, -0.0039317645, -0.018777339, 0.01466219,
0.018047577, 0.017742017, -0.00015193067, -0.0006078845, 0.020327229,
-0.0031070304, -0.015753543, 0.0055653704, 0.006133356, 0.0012707526,
-0.0010969826, -0.014554295, -0.0018951239, -0.011392781,
-0.006888789, 0.007748293, 0.0035578525, 0.003289515, 0.012910731,
-0.00093347987, 0.027975185, -0.021436982, -0.018103244, 0.014160516,
0.0043639764, 0.0036033795, -0.01316905, -0.007073482, 0.0071901446,
-0.0035180696, 0.011319165, -0.005156143, 0.020395549, 0.014424411,
-0.013264267, -0.008368782, -0.006329851, 0.01564824, -0.03172509,

0.027850118, -0.0024029233, -0.021380346, -0.006108456, 0.0027379538,
-0.005856265, -0.0016397117, 0.0006714314, 0.012711725, -0.008265198,
-0.023876442, -0.038576953, -0.0069513926, 0.01963346, -0.027351905,
-0.0002880972, 0.0016497897, -0.011469009, -0.0076140366,
-0.011518238, 0.0148204155, -0.014008247, 0.011123518, 0.01799503,
-0.008001081, 0.012425253, -0.008350129, 0.0064347233, -0.019037358,
0.0045397435, 0.017119428, -0.01512124, -0.006468074, -0.024882043,
0.013289902, 0.007395278, 0.029273696, 0.007908903, -0.0033503755,
0.01254286, 0.016428003, 0.0037401575, 0.0050791567, -0.0036261447,
0.0140235, -0.008065762, -0.014647894, 0.012400106, -0.0052588293,
0.010449422, 0.005727772, 0.010904326, -0.0061714277, 0.013405642,
-0.01387189, -0.0012836957, -0.013784845, -0.0033544423, 0.008126222,
0.009253681, 0.0014313303, 0.002983682, -0.007894025, -0.012918289,
-0.0077993353, 0.03520822, -0.027082797, -0.0099341255,
-0.0015001617, -0.0013335982, -0.007475705, 0.0017566493,
0.0050008926, -0.021517286, -0.0011548921, -0.0029852001, 0.01520633,
5.5708293e-05, -0.01033985, -0.0035706067, 0.0030326913, 0.020252807,
-0.009566282, -0.010538089, 0.0014786252, -0.015273831, -0.003915893,
-0.0063233753, -0.015660513, -0.015529271, -0.0072143287,
-0.008542986, -0.008686909, 0.016915258, -0.004854617, 0.006381227,
-0.012484433, -0.033222772, -0.0054721707, 0.00879257, -0.0024218082,
-0.0097658, 0.02203544, 0.008975894, -0.006289271, 0.0096442895,
0.009777785, 0.00838704, -0.020963686, 0.021196926, 0.014794346,
0.0025145963, 0.030052708, -0.018815318, -0.009033906, 0.0035851982,
-0.0063334745, -0.022092987, -0.006286732, 0.0347471, 0.022634199,
-0.031376973, 0.012749768, 0.015011687, 0.0013633412, -0.007360656,
0.01592488, -0.008927109, -0.008928848, 0.02620737, -0.010186921,
0.015393375, 0.002507383, -0.026446821, -0.001740027, -7.6487355e-05,
-0.0015983396, 0.012287021, -0.020735852, 0.018017976, -0.012436571,
0.0082112225, 0.004855035, 0.01776712, -0.0029890458, 0.006013396,
-0.02202631, -0.012203404, 0.010941057, -0.012564804, -0.00025892578,
0.015001843, -0.010635011, 0.00031877204, -0.0010992913,
-0.030433593, 0.0049771825, -0.0067421794, 0.016350737, -0.005100247,
-0.011224739, -0.0047967387, -0.021097586, -0.008989857,
-0.019488156, -0.0051507107, 0.0019571958, -0.015338324,
-0.0015634885, 0.005515723, 0.022735199, 0.0068165124, 0.015280049,
0.010241081, -0.005332229, 0.020268615, 0.009170835, 0.003864846,
-0.004640622, 0.0060196966, -0.028362479, -0.003389505, -0.009913119,
0.007278818, -0.018089138, -0.02342628, -0.03654425, 0.02233085,
-0.02019963, 0.012823049, 0.0028359822, 0.0046371217, 0.0019381484,
-0.009071803, -0.010917661, -0.015543864, 7.8957266e-05,
-0.018822651, -0.017599083, 0.006820304, -0.0072462633, 0.007964995,

-0.012614167, 0.0045171734, 0.0040539023, -0.015114458, -0.008019503,
-0.016230257, -0.0030027828, -0.014588336, 0.020485578, 0.011333489,
-0.0009879809, 0.0064244308, 0.011697777, 0.016982913, 0.010120816,
-0.014851334, -0.01615345, -0.023203675, -0.0039236294, 0.0009910885,
-0.0019104262, 0.00032107555, -0.0038652185, -0.0075321607,
-0.015767375, 0.00054077763, 0.0078757275, -0.032672524,
-0.0076399804, 0.0030912093, 0.018132474, 0.013199343, -0.009583627,
-0.0012999396, 0.015379879, -0.006070005, -0.020560713, 0.0064747236,
-0.0074686324, -0.010605387, -0.0023784472, 0.005996246,
-0.004229647, 0.012629265, 0.010708814, 0.013211182, -0.006557063,
0.001812954, -0.014146787, 0.009376801, -7.9025915e-05, -0.019875605,
0.0030873527, -0.0120186405, -0.007535565, -0.0009900556,
0.008227048, -0.0037357067, 0.011779844, -0.0015810898, 0.006049016,
-0.00228235, -0.0016605607, 0.024967866, -0.009355245, 0.0012407538,
0.0031669384, 0.0003603365, -0.019062467, -0.00775442, -0.012790427,
0.00835641, -0.0049819998, 0.0047103846, 0.0006130769, -0.008858576,
-0.027195973, -0.01994057, 0.012000719, -0.008283194, -0.011214109,
-0.012306973, -0.0029980668, 0.024302179, -0.0034042457, -0.03701426,
0.0019187469, 0.012065776, -0.0014142025, -0.0021428654,
-0.018199395, 0.016543116, -0.012012498, -0.0016578208, -0.017312467,
0.011349779, -0.0012463382, 0.027023418, 0.001488224, -0.0006933667,
0.014585088, -0.015000108, 0.025585195, -5.5853943e-05, 0.013409394,
0.0032994058, -0.018429236, -0.011921773, -0.0049212887, 0.011364606,
-0.012736779, 0.037215907, 0.022860777, -0.0046487865, 0.0060092183,
0.019911185, -0.011840368, 0.009407341, 0.016642591, -0.0059299227,
-0.023696695, -0.006275238, -0.009079175, 0.019764392, -0.012575573,
0.0047461153, 0.015107097, -0.013777879, 0.0023578394, -0.007894431,
0.0007694715, 0.024754329, 0.0011582341, 0.020207357, 0.010338741,
0.0101574855, 0.0056565353, 0.006682724, 0.0077401125, 0.0132401725,
-0.0063025965, 0.017188074, 0.030116487, 0.014875936, -0.021115694,
-0.0063506705, 0.022894202, 0.019081004, -0.010526954, -0.011320607,
-0.020598235, 0.015328102, -0.007721765, -0.014667402, -0.007438127,
-0.011928842, 0.0007742271, 0.013745593, 0.030018385, -0.051551502,
0.015579759, -0.030218797, -0.017032893, -0.00423585, -0.006663313,
0.021118272, -0.009510693, -0.011373037, -0.02507755, 0.01108873,
0.012970266, 0.013108182, -0.007240811, -0.000825215, 0.008814925,
0.0054254364, -0.012955095, -0.018055929, -0.011119609,
-0.0034894294, -0.0051534637, -0.004303848, 0.00078492257,
0.003636195, 0.00019605862, 0.00334298, 0.016054235, -0.014453912,
0.0061524515, -0.01019406, 0.018490428, -0.018525854, -0.0028730396,
0.0006314976, 0.019112797, 0.0039287833, 0.025050264, -0.0007321807,
-0.017491417, 0.0045831623, -0.010098036, 0.0016023709, -0.004088477,

0.0062807193, 0.009442158, -0.015026809, 0.005650422, 0.006104784,
0.01970638, -0.0054841824, 0.0028440703, -0.011502451, 0.0066077933,
-0.0103631355, -0.0033619653, 0.0026373875, 0.01697218, 0.011431076,
0.00815383, -0.010191866, -0.031792782, -0.009102554, 0.014624937,
-0.023171194, -0.003782474, 0.035593428, 0.0069384193, 0.0079221735,
-0.002558078, -0.010756514, 0.0060175327, -0.006732807,
-0.0011438341, 0.06781527, -0.003156085, -0.018987296, -0.013228836,
0.007675978, 0.0006253307, -0.010466165, 0.027876034, -0.016284743,
0.003529613, 0.008411483, -0.004632434, 0.008429623, 0.0033291231,
0.00068884954, -0.006476403, -0.01643092, -0.033781394, -0.016462142,
-6.2131694e-05, 0.0043627056, 0.009406918, -0.009169348,
0.0027015572, -0.005546772, 0.008620843, 0.008132464, -0.013531863,
0.024464918, 0.007398241, 0.0019932485, 0.024615576, -0.019378629,
-0.012897572, 0.014438142, -0.0031768312, -0.02300522, -0.0113402335,
-0.01090301, -0.0035319855, 0.0020362441, -0.006237794, 0.004456702,
0.012718032, 0.015141308, 0.0077613806, -0.023979452, -0.028067201,
0.023297047, -0.026084227, 0.0016661165, 0.015354106, -0.005276987,
-0.00045021824, 0.0020882029, 0.0073567294, -0.014785464,
-0.023275793, 0.0032892716, -0.0016042996, 0.0059375833,
-0.0011868294, -0.0037843434, 0.0137570845, 0.012237218,
-0.011657946, -0.008400644, 0.023069413, -0.0058594123, -0.018971493,
-0.00080640183, -0.02052293, 0.01361178, 0.01279665, 0.020574559,
-0.018910091, -0.0039544785, 0.00801305, 0.015138, 0.012626505,
0.00419911, 0.003416181, -0.001947856, -0.0029749891, -0.0059151566,
0.01062549, -0.015961226, 0.027512662, -0.015969016, 0.0015483764,
0.015715994, -0.0051350025, 0.0134951165, -0.022569872, 0.0075369137,
0.0022642948, 0.00077266357, -0.015832303, -0.00201792, 0.014555646,
-0.00987233, -0.0057685142, 0.0057743206, 0.003392713, 0.0037948408,
0.016472105, -0.007836597, 0.012416375, -0.008207014, 0.013171638,
0.012602578, 0.0055397945, 0.020464139, -0.00206017, 0.018007383,
0.0047173235, 0.017250037, 0.0031103226, 0.004787774, -0.03059456,
0.01625869, -0.0053125415, 0.034866143, -0.003688748, 0.021133756,
-0.007874794, -0.02127866, 0.0047416985, 0.013142121, 0.028185964,
0.010764741, -0.014586055, -0.008590581, 0.0077432008, 0.018469734,
-0.0065713758, -0.016032968, 0.00825615, -0.0007354428, -0.012345303,
0.022020673, 0.018086167, 0.18199897, 0.122733854, -0.0071706716,
-0.013163544, 0.026301444, 0.005301678, 0.0045072497, -0.016251948,
-0.005219496, -0.015062984, 0.0028787258, 0.0034464558, 0.014252049,
-0.018672224, -0.00339187, 0.0024929703, 0.00051383756, 0.0063335574,
0.014966912, -0.0068134763, 0.008938959, 0.0130825145, -0.022433715,
-0.007844837, 0.009103279, 0.0077576195, 0.0036069292, 0.014116835,
0.012400754, -0.010279591, 0.00011030357, 0.006943821, 0.008722375,

-0.009385252, -0.00972463, -0.012826249, -0.005934716, -0.017518526,
-0.012132905, -0.0011956048, -0.020564804, 0.013111334, -0.010754355,
-0.0031549137, -0.011284525, 0.010522077, -0.00848092, -0.0021737842,
0.006816838, 0.031295005, 0.009359975, 0.00034741027, 0.015510683,
-0.012714861, -0.010075302, -0.010084895, -0.0050249044, 0.016593015,
-0.010575505, 0.0003504634, 0.015356935, 0.0075303772, -0.01633746,
-0.016149143, 0.005326476, 0.010200405, 0.0055319956, 0.007638612,
0.007130913, -0.009724308, 0.009432359, 0.016036853, 0.00064382795,
0.017268816, -0.006427538, 0.002043632, 0.023802828, 0.010041443,
-0.0050353673, -0.012472469, -0.018143082, 0.0037194083,
-0.006216387, -0.0058859647, 0.010636436, 0.008984544, -0.0009132329,
0.040118437, 0.14963067, 0.0056761275, -0.012803241, -0.03177453,
-0.0045894645, 0.01661909, -0.0027402735, 0.049565766, 0.009954993,
0.00020866856, -0.007877462, 0.007004008, 0.0136196045, 0.0065690395,
0.008130257, -0.025948435, 0.020605078, 0.051170714, -0.007738707,
-0.0010499568, -0.0014188697, -0.009419122, -0.0028952614,
0.012380499, 0.0057063275, 0.0049448013, 0.008131117, 0.002107506,
-0.01107746, 0.014290169, -0.06882972, 0.008140089, -0.008393951,
-0.008240544, 0.02619541, -0.007345089, 0.010071826, -0.015133152,
0.00065869984, 0.01344989, -0.0056055128, -0.015576932, -0.006644164,
0.00024300796, -0.025651405, -0.0075896326, -9.470708e-05,
-0.015156963, 0.0033765172, 0.023837544, 0.01234019, 0.0020032274,
-0.0011578423, 0.009669884, 0.0073583014, -0.011883421, -0.005666323,
-0.0036043052, 0.015326066, -0.008676161, -0.012866116, -0.00806899,
-0.0047081816, -0.0014456876, 0.013877029, 0.0107890135,
-0.010754017, 0.009919641, 0.018208086, -0.0125982035,
-0.00019057764, -0.036416214, -0.01862857, -0.011150162,
0.0062048407, -0.008039027, 6.3353327e-06, -0.0060262885,
0.003258782, -0.0027400614, 0.04300614, -0.021198608, -0.0033233815,
0.005844234, 0.013522905, -0.020132037, 0.0010979873, 0.008765486,
-0.009159733, -0.0011814921, 0.00064021273, 0.00048474892,
0.0059410688, 0.0013943472, -0.009121826, 0.0024298704, 0.0039277906,
0.014253299, 0.0014913729, 0.0040087346, -0.007813209, 0.013938316,
-0.016969388, -0.013650684, 0.0073228255, 0.002765821, -0.016812976,
-0.008476319, 0.011485381, -0.0043964996, 0.008805819, 0.0030225164,
0.023460917, 0.13892579, -0.005332994, 0.0063376464, 0.009567981,
-0.0044250838, 0.0080510955, -0.012685185, 0.004658339,
-0.0031256056, 0.019114159, 0.001748571, -0.016904293, 0.0066619627,
-0.0175385, 0.0048648976, 0.020754654, -0.0039505525, -0.0052116257,
-0.024088493, -0.0110295685, -0.010888556, -0.013623145,
0.0059099686, -0.0045290855, 0.00630125, -0.0017370215,
-0.0019653833, 0.010573742, 0.0016552886, 0.015912155, -0.0020647564,

0.0075304154, -0.001886199, -0.021219857, -0.0016973737,
-0.0007540455, -0.018016845, -0.023638917, 0.0122922, -0.011445283,
0.008122043, 0.008397747, -0.018533787, 0.0070802923, -0.0176786,
0.19236869, -0.006474825, 0.023981871, -0.020533433, 0.018786082,
0.018690443, -0.008969684, 0.023202596, 0.010060016, -0.013235595,
-0.0061381506, -0.00022583171, 0.0042697866, -0.00016152389,
-0.009365772, -0.007523141, -0.0020800314, -0.022448845,
-0.019692097, 0.015373903, -0.02021986, 0.0046861274, 0.022467515,
-0.002851628, 0.019820422, 0.019209418, 0.007827799, 0.02006786,
0.007436729, -0.0031383769, 0.009567939, 0.009463483, 0.008926504,
0.008760794, -0.0246795, 0.011080253, -0.0036806841, 0.010065379,
-0.006923873, -0.0065189255, 0.01687139, -0.016856616, 0.03498424,
-0.013988072, 0.0012248241, 0.002965315, -2.7035585e-05,
-0.010670625, -0.004798875, 0.012956593, -0.00052243733, -0.03831337,
0.011291657, -0.006973239, 0.005756488, -0.026281115, 0.00890928,
-0.0013892314, -0.026096502, 0.02019696, 0.0007254554, 0.0014532299,
-0.0083217425, -0.003224226, 0.009292625, 0.017529177, -0.0013907467]